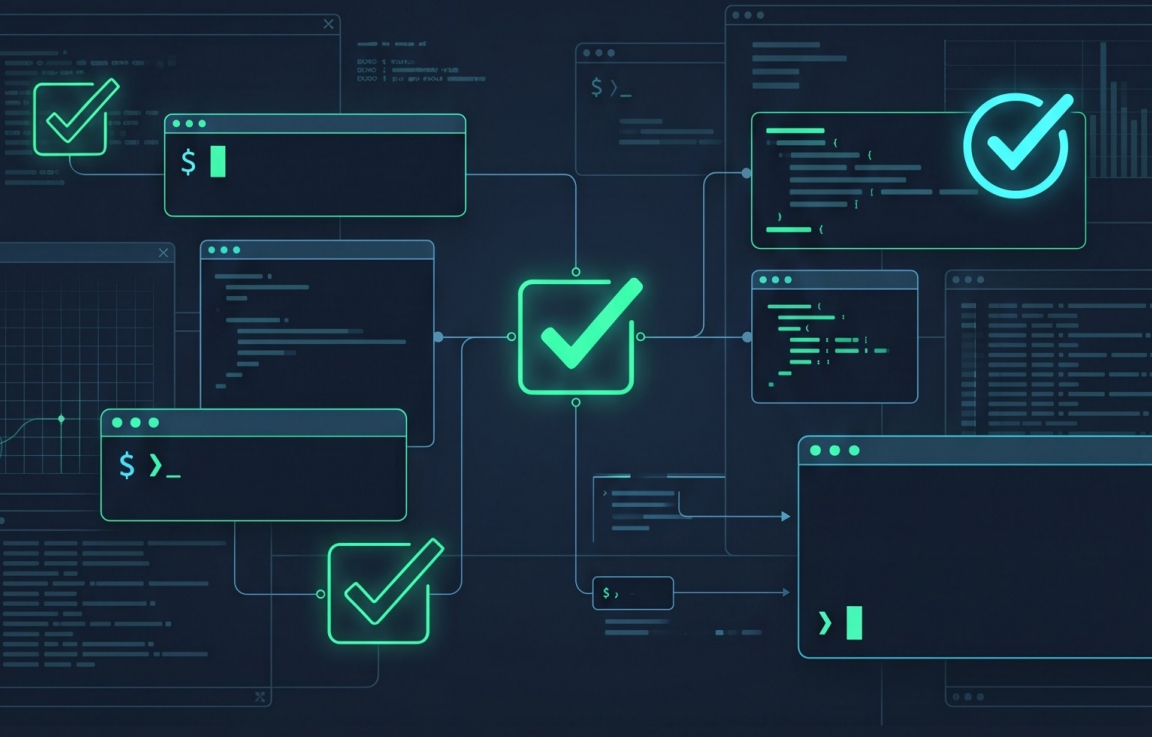
Desenvolvimento Verificável com Agentes de Código

Evidências, Spec-Driven Development e TDD no terminal



Regis Araujo Melo

Aviso editorial

© 2026 Regis Araujo Melo. Todos os direitos reservados.

Nenhuma parte desta obra pode ser reproduzida ou distribuída sem autorização por escrito do autor, exceto em citações breves com indicação da fonte.

Comandos, nomes de skills e opções das CLIs foram conferidos em outubro de 2026 e podem mudar em versões posteriores. Confira a documentação da sua CLI antes de executá-los.

As informações são fornecidas sem garantia. Revise todo comando e todo código antes de executá-los no seu ambiente.

Claude Code é marca da Anthropic. GitHub Copilot é marca da GitHub, Inc. Os nomes identificam os produtos e não indicam afiliação nem endosso.

Sumário

Introdução 6
Para quem é este livro 6
A tese 6
O que você vai conseguir fazer 7
Como o livro está organizado 8
Como ler 9
Convenções 9
Pré-requisitos 10
Capítulo 1 — Rápido não é verificável 11
Um pedido vago 11
Três falhas típicas 12
Testes tautológicos 13
O custo deslocado para o fim 14
O princípio zero-trust 15
Desenvolvimento verificável 16
Do princípio ao processo 17
Resumo 17
Capítulo 2 — Agente = modelo + harness 19
O harness é um loop de controle 19
A superfície de ferramentas 20

Permissões como fronteira de confiança 21
Degradação de contexto 22
Por que o agente inventa APIs 24
MCP: capacidade e superfície de ataque 25
Prompt, ferramenta e skill 25
Resumo 26
Capítulo 3 — Skills: um processo em arquivo 28
Skill é processo versionado 28
Divulgação progressiva: orçamento de contexto 29
Acionamento não é determinístico 30
Onde as skills moram 31
Padrões de engenharia com IA 31
Instalando o superpowers 33
Skills de terceiros: risco de cadeia de suprimentos 34
Resumo 35
Capítulo 4 — Evidência e Spec-Driven Development 37
Seis evidências, seis gates 37
SDD: engenharia de requisitos executável 39
Critérios com ID: a base da rastreabilidade 40
A curva de custo da correção 41
Spec Kit e a constituição 42
O fluxo mínimo nos padrões canônicos 43
Variação para código legado: o passo zero 44
Antes da Parte II 45
Resumo 45
Capítulo 5 — Esclarecer 47
Por que o modelo preenche lacunas 47
O padrão e a sua implementação de referência 48
Uma taxonomia de lacunas 48
Quem decide o quê 49
O mínimo de condução 50
A spec como estado fora do contexto 51
Auditando a spec 52
Erros comuns 53
Resumo 53
Capítulo 6 — Planejar 55
Decomposição Verificável 55
O plano como contrato 56
Em código legado: caracterizar antes, extrair depois . . . 56

Portas e injeção como pré-condição de testabilidade . . . 58
A lacuna que o plano descobre . . . 58
Auditar o plano contra os IDs . . . 59
Erros comuns . . . 60
Resumo . . . 61
Capítulo 7 — Vermelho, depois verde . . . 62
Prova por Falha . . . 62
Falhou ou quebrou: o vermelho pelo motivo certo . . . 63
Teoria do teste de mutação . . . 63
Quebrar de propósito: a mutação manual . . . 66
O vermelho da caracterização . . . 67
Desconfie do vermelho "consertado" no teste . . . 67
Portão de Verificação Independente . . . 68
A evidência final: a suíte e o diff . . . 68
Erros comuns . . . 69
As seis evidências, reunidas . . . 69
Resumo . . . 70
Capítulo 8 — Escala, limites e próximos passos . . . 71
Escalar é isolar estado . . . 71
A evidência de segunda mão . . . 73
CI/CD: o pipeline como guardião zero-trust . . . 74
Revisão Independente e o fechamento da branch . . . 76
Limites . . . 77
Código legado e integração . . . 78
Numa história real . . . 79
Os padrões sobrevivem às ferramentas . . . 80
Resumo . . . 80
Apêndice A — Mapa das skills . . . 82
Fluxo mínimo . . . 82
Escala . . . 82
Revisão e entrega . . . 83
Depuração . . . 83
Skills sobre as skills . . . 83
Sem skill própria . . . 83
Apêndice B — Glossário . . . 84
Apêndice C — Equivalências entre as CLIs . . . 92
Instalar o pacote como plugin . . . 92
Instalar só algumas skills . . . 92
Pastas de skills . . . 92

Conferir e chamar 93
Apêndice D — Projeto do exemplo 94
Estrutura e dependências 94
Ponto de partida 94
Ponto de chegada 95

Introdução

Um *agente* de IA produz em segundos um diff que levaria uma tarde para ser escrito. Isso não é o assunto deste livro. O assunto é o que vem depois: como saber que aquele diff está certo, antes do merge e não depois do incidente. As próximas páginas apresentam um método para exigir *evidência* em cada etapa do trabalho com o agente.

Para quem é este livro

Este livro é para engenheiros plenos e seniores, tech leads e arquitetos que já usam agentes de código ou que vão definir como o time deve usá-los. Ele supõe domínio de Git, CI/CD, TDD, testes automatizados e arquitetura modular, e não para para explicar nada disso. O exemplo é em TypeScript, mas nenhuma parte do método depende da linguagem.

Os exemplos usam duas *CLIs*, o GitHub Copilot CLI e o Claude Code. Elas são veículos, não o tema. Os comandos específicos ficam concentrados no Capítulo 3 e no Apêndice C; o resto do livro vale para qualquer agente que leia arquivos e rode comandos.

Se é você quem assina o pull request ou responde pelo serviço quando ele quebra, a pergunta "como eu sei que isto está certo?" já é sua. O livro trata de respondê-la quando parte do código foi escrita por um agente.

A tese

A tese cabe numa frase: toda saída do agente é não confiável até ser verificada por um mecanismo que não dependa do próprio agente. É a postura *zero-trust*, trazida da segurança de redes para o desenvolvimento com IA. Numa rede zero-trust, estar do lado de dentro não concede confiança. Aqui, uma resposta fluente, bem formatada e segura de si também não concede.

Isso não é pessimismo; é uma descrição da ferramenta. O agente é falho por construção: gera o texto mais plausível, não o verificado, perde instruções em sessões longas, inventa APIs e declara pronto o que não conferiu. O Capítulo 2 mostra de onde vêm essas falhas. A confiança, portanto, tem de estar em mecanismos: um teste que você viu falhar, um comando que você rodou, um gate de CI que barra o merge. Velocidade não entra nessa conta; entregar rápido o que ninguém consegue

verificar só desloca o custo.

A segunda ideia é a de padrões. O trabalho se organiza em *padrões de engenharia com IA*, formas recorrentes de restringir o agente para que ele produza artefatos verificáveis: esclarecer o requisito antes de codar, decompor o trabalho em passos conferíveis, provar a mudança com um teste que falha primeiro, verificar de forma independente antes de declarar pronto, isolar contexto e espaço de trabalho. No pacote aberto superpowers, quase todos os padrões têm uma *skill* que os implementa. A skill é a *implementação de referência*; o padrão é o que você leva para outra CLI, outro pacote ou um processo do próprio time. A Parte I dá nome a cada um.

O que este livro não é:

- não é sobre o funcionamento interno dos modelos; o Capítulo 2 trata só do que afeta a verificação;
- não é um catálogo de prompts prontos;
- não é um tutorial de ferramenta: CLIs e pacotes de skills mudam rápido, e o livro os usa como veículos;
- não é uma introdução à engenharia de software: TDD, revisão e CI são pressupostos; o livro mostra o que muda neles quando o autor do código é um agente;
- não é sobre agentes autônomos sem pessoa no circuito. O método supõe alguém que aprova a spec, o plano e o diff. O Capítulo 8 trata de escala e de CI de verdade, com gates obrigatórios no pipeline, e mantém a revisão humana como um deles.

O que você vai conseguir fazer

Ao terminar o livro, você vai conseguir:

- **aplicar** o princípio zero-trust: separar afirmação de evidência e exigir, para cada saída do agente, um mecanismo de verificação que não dependa dele;
- **explicar** o agente como *modelo + harness* e, a partir disso, as falhas que você precisa prever: degradação de contexto e alucinação de API;
- **reconhecer** os padrões de engenharia com IA e as skills que os implementam, e avaliar uma skill de terceiros como código que roda com as suas permissões;

- **conduzir** uma história pelo fluxo mínimo, nesta ordem: esclarecer, planejar, ver o teste *vermelho*, deixá-lo *verde* e verificar;
- **proteger** código legado com *testes de caracterização* antes de deixar o agente mexer nele;
- **converter** cada evidência num gate de pipeline e decidir quando o fluxo completo vale o custo.

Cada etapa do método deixa um artefato que outra pessoa pode inspecionar sem confiar em você nem no agente. Esse artefato é a evidência. É ele que separa "o agente disse que terminou" de "está demonstrado que está pronto".

São seis evidências, duas para cada momento do trabalho:

- **Antes do código:** requisito esclarecido (a *spec*) e *plano* revisável.
- **Durante:** teste falhando antes do código e teste passando depois.
- **Depois:** *diff* revisado por você e resultado verificado por você.

Em código legado há um pré-requisito antes de todas elas: a linha de base caracterizada, que registra o comportamento atual antes de qualquer mudança. O Capítulo 4 mostra onde fica cada evidência e qual gate automatizado corresponde a ela, e a Parte II produz as seis no exemplo.

Como o livro está organizado

O livro tem três partes e quatro apêndices. No ebook, cada título abaixo é um link.

Parte I — Fundamentos. O problema, o princípio e as peças: por que velocidade não é verificação, o que é um agente, o que é uma skill e como cada evidência vira um gate.

- Capítulo 1 — Rápido não é verificável
- Capítulo 2 — Agente = modelo + harness
- Capítulo 3 — Skills: um processo em arquivo
- Capítulo 4 — Evidência e Spec-Driven Development

Parte II — Na prática. O método aplicado a uma história com código legado, contrato de interface e uma dependência de outro serviço, do pedido inicial até o teste passando e a mutação conferida.

- Capítulo 5 — Esclarecer
- Capítulo 6 — Planejar

- Capítulo 7 — Vermelho, depois verde

Parte III — Além do primeiro teste. Escala com subagentes e worktrees, o pipeline de CI que torna as evidências inegociáveis e os limites do método.

- Capítulo 8 — Escala, limites e próximos passos

Apêndices. Consulta rápida para depois da leitura.

- Apêndice A — Mapa das skills
- Apêndice B — Glossário
- Apêndice C — Equivalências entre as CLIs
- Apêndice D — Projeto do exemplo

Como ler

Na primeira leitura, siga a ordem dos capítulos. A Parte II é um roteiro de método e pressupõe a Parte I; lida sozinha, mostra os passos sem o critério que justifica cada um. Nela, o exemplo aparece só em caixas curtas; o código completo e verificado está no Apêndice D, e é com ele que você reproduz os vermelhos do Capítulo 7.

Quem está desenhando o pipeline do time pode ir do Capítulo 4 direto ao Capítulo 8. Depois da primeira leitura, a Parte I serve de referência para justificar o processo diante do time.

Convenções

Ao longo do livro, quatro tipos de caixa destacam informações:

> **[Nota]** Contexto extra ou detalhe que ajuda a entender o texto, mas que você pode pular numa leitura rápida.
>
> **[Dica Prática]** Um hábito ou ajuste que reduz atrito no dia a dia com o agente.
>
> **[Aviso de Erro Comum]** Um erro recorrente, com o sintoma e a forma de evitá-lo.
>
> **[Exemplo]** Um caso concreto, em código ou em texto, que mostra a ideia aplicada.

Código e comandos aparecem em `fonte monoespaçada`, no meio da frase ou em blocos separados. Quando o GitHub Copilot CLI e o Claude Code fazem a mesma coisa de formas diferentes, há dois blocos, um rotulado *Copilot CLI* e outro *Claude Code*; isso acontece quase só no Capítulo 3 e no Apêndice C.

Termos técnicos aparecem em *itálico* na primeira vez em que são usados. Todos estão definidos no glossário (Apêndice B).

Pré-requisitos

Para reproduzir o exemplo:

- acesso a um agente de código com ferramentas de arquivo e de terminal; os exemplos usam o GitHub Copilot CLI ou o Claude Code;
- Node.js em versão LTS, para rodar o projeto de referência do Apêndice D;
- um repositório em que o agente trabalhe numa branch local, com aprovação de comandos ligada e sem segredos, tokens ou credenciais de push e de deploy ao alcance dele.

O último item é a primeira aplicação da tese: o que o agente não consegue fazer não precisa ser verificado depois.

> **[Nota]** As CLIs de IA mudam rápido. Os comandos deste livro foram conferidos na data de publicação, mas podem mudar de nome ou de forma. O método não depende deles: se um comando mudar, o padrão, a etapa e a evidência que ela produz continuam os mesmos.

Capítulo 1 — Rápido não é verificável

Parte I — Fundamentos

Gerar código ficou barato; avaliar, não. Saber se uma implementação respeita o requisito, os contratos existentes e as invariantes do sistema custa o mesmo de antes, quem quer que a tenha escrito. Este capítulo mostra onde esse custo se esconde quando ninguém o planeja, formula o princípio que guia o resto do livro e define o que conta como evidência.

Um pedido vago

O livro acompanha uma única história, num sistema com o tipo de restrição que torna a verificação difícil: código legado, um contrato que não pode quebrar e uma dependência de outro serviço.

O sistema é o `transfer-service`, da plataforma de transferências do Banco Azul. O módulo legado `src/legacy/fees.ts` expõe `getFee(code, amount)`, que lê uma tabela global carregada na inicialização e substituída pela sincronização noturna de preços. Não tem testes, e o serviço de extrato depende dele: a assinatura e o comportamento de `getFee` são um *contrato de interface*. Cada banco tem duas tarifas, uma para valores até R$ 1.000,00, inclusive, e outra para valores acima disso. No Banco Azul, R$ 2,00 e R$ 5,00. O legado aplica essa regra corretamente.

Há também uma fonte nova de tarifas. A porta `FeeTableProvider` busca a tabela no pricing-service, outro microsserviço, e a mantém em cache com refresh. Duas chamadas seguidas a `current()` podem devolver versões diferentes da tabela.

A história é o ranking de bancos na tela de transferência: a pessoa digita um termo e um valor, e o serviço devolve os bancos correspondentes, por nome ou por código, ordenados pela tarifa daquele valor. A exigência de arquitetura é uma só: a tarifa exibida no ranking tem de ser a mesma que o extrato cobra.

Agora o pedido, do jeito que ele costuma ser feito:

```
Crie o ranking de bancos para a tela de
transferência. Filtre pelo termo digitado e
ordene pela tarifa do valor informado, usando
a tabela de tarifas do pricing.
```

Para quem conhece o sistema, o pedido parece suficiente. Ele não diz se o termo casa com o código, qual regra de faixa vale nem quantas leituras da tabela uma resposta pode fazer, e não menciona o extrato. O agente responde em segundos com um `BankRankingService`, um arquivo de testes e uma mensagem segura. Nada na resposta diz se o código está certo, e ele não está.

Três falhas típicas

As três falhas não são exclusivas de agentes. O agente só as torna mais frequentes, porque nunca hesita: toma cada decisão com a mesma fluência, tenha ou não base para ela.

Implementar antes de esclarecer O pedido tinha lacunas, e o agente as preencheu em silêncio. A primeira é de negócio: a busca compara o termo só com o nome, e quem digita "001" não encontra o Banco Azul. A segunda é de contrato e de concorrência, e é mais grave.

[Exemplo] O núcleo do serviço gerado:

```
for (const bank of matches) {
  const table = await this.tables.current();
  const range = table.fees[bank.code];
  const fee = amount < table.limit
    ? range.lowFee
    : range.highFee;
  ranked.push({ ...bank, fee });
}
```

A chamada a `current()` dentro do laço faz uma leitura por banco. Com três bancos, são três leituras por requisição. Se o cache renovar no meio do laço, a mesma resposta mistura duas versões da tabela, e o ranking ordena tarifas que nunca coexistiram. Nenhum teste com provider fixo enxerga isso, e o defeito só aparece em produção, perto do refresh: intermitente, dependente de tempo e sem relação visível com a mudança que o introduziu.

O código não está mal escrito. Ele implementa com competência uma decisão que ninguém tomou. Havia uma pergunta a fazer, "uma resposta pode ler a tabela mais de uma vez?", e ela não foi feita.

Escrever o teste depois, tautológico O mesmo trecho tem outra decisão silenciosa. Em vez de reaproveitar a política de tarifa do legado, o

agente a reimplementou no serviço novo, com `amount < table.limit`. No limite exato, R$ 1.000,00, a expressão cai na faixa alta. O legado, com `amount > limit`, cai na baixa.

Depois de escrever o código, o agente escreveu o teste:

> **[Exemplo]** O teste gerado afirma o que o código faz:
>
> ```
> it("charges 5 for 1000", async () ⇒ {
>   const r = await service.rank("azul", 1000);
>   expect(r[0].fee).toBe(5);
> });
> ```
>
> A tela mostra R$ 5,00. O extrato cobra R$ 2,00.

É um *teste tautológico*: passa por construção, porque o valor esperado veio da implementação, não do requisito. Um teste de caracterização do legado, afirmando que `getFee("001", 1000)` devolve `2`, teria exposto a divergência no primeiro vermelho. O Capítulo 4 volta a esse passo zero.

Dizer "pronto" sem evidência A resposta termina assim: "Pronto! Implementei o ranking com busca e ordenação por tarifa. Os testes devem passar."

"Devem passar" não é "passaram". O agente não rodou nada: previu o resultado e o apresentou no tom de quem conferiu. O modelo não marca a diferença entre o que observou e o que inferiu.

E, mesmo que tivesse rodado a suíte, o verde não provaria nada sobre a tarifa, porque o teste era tautológico. Rodar o teste é necessário e não é suficiente. Evidência exige as duas coisas: um teste que mede o requisito e uma execução que você observou.

> **[Aviso de Erro Comum]** Tratar a fluência da resposta como sinal de correção. A confiança do texto não mede a qualidade do código. O sintoma é aprovar porque "a explicação fazia sentido". Troque a pergunta "isto parece certo?" por "que mecanismo, fora do agente, mostrou que isto está certo?".

Testes tautológicos

Um teste tautológico é aquele cujo valor esperado deriva da implementação, e não do requisito. A forma mais comum é a do exemplo: o mesmo

autor escreve o código e, em seguida, o teste, com a mesma compreensão do problema. Se ela estiver errada, os dois erram juntos, e o verde só confirma que concordam entre si.

Há variações, como afirmar que um mock foi chamado em vez de afirmar o efeito, ou copiar para o `expect` o valor que a primeira execução imprimiu. Todas têm a mesma estrutura: o oráculo do teste não é independente do código testado.

Três propriedades o tornam perigoso, e não apenas inútil:

- **Ele é invisível às métricas usuais.** Cobertura mede o que foi executado, não o que foi verificado. O teste do exemplo cobre o ramo e afirma o valor errado.
- **Ele congela o erro.** Quem corrigir a regra depois verá esse teste ficar vermelho e terá motivo para achar que a correção é que quebrou algo.
- **Ele fica mais provável com agentes.** Código e teste saem da mesma sessão, com o mesmo contexto. Sem restrição de processo, os erros dos dois ficam correlacionados.

Um teste só protege algo se for escrito a partir do requisito e se você o vir falhar pelo motivo certo antes de existir o código. Como medir se uma suíte protege o código, e não apenas o executa, é assunto do Capítulo 7.

O custo deslocado para o fim

Nenhuma das três falhas atrasou a entrega. O custo não desapareceu; mudou de lugar. Sem processo, toda a verificação se concentra no fim, sobre um diff já escrito, e lá enfrenta três problemas de engenharia que nenhuma atenção extra resolve.

Acoplamento entre consumidores. A regra de tarifa tinha um dono, o legado, e um consumidor, o extrato. O agente criou um segundo dono. Agora há duas fontes de verdade para a mesma regra, e cada uma passa nos próprios testes. A divergência não existe dentro de nenhum dos módulos; existe na fronteira entre eles. Quem a vê é o cliente, quando a tela promete R$ 5,00 e o extrato cobra R$ 2,00, ou a conciliação, semanas depois. Regra duplicada é acoplamento implícito: os dois lados precisam mudar juntos, e nada no código registra isso.

Contrato quebrado sem que nada falhe. O comportamento de `getFee`

no limite é parte do contrato, mas não está escrito em lugar nenhum além do próprio código. O mesmo vale para a semântica de `current()`: duas chamadas podem divergir, e quem consome a porta precisa decidir quantas leituras uma operação faz. Contratos implícitos não quebram com erro de compilação nem com teste vermelho; quebram em silêncio, longe da causa.

A revisão no fim não reconstrói decisões. Um diff mostra o que foi feito, não o que foi decidido nem o que foi deixado de fora. O revisor confere o que está presente; a pergunta que ninguém fez não aparece em linha nenhuma. Diante de um diff grande, coerente e com testes verdes, a revisão tende a virar conferência de estilo, e escapam justamente os erros que parecem corretos. Quanto mais código o agente produz, mais esse gargalo pesa.

O custo de corrigir também cresce com a distância. Uma pergunta antes do código custa segundos. O mesmo erro depois do merge exige achar a causa, corrigir a regra, desfazer o teste que congelou o erro e auditar quem passou a depender do comportamento errado. O agente acelerou a parte barata, que é escrever, e empurrou a cara, que é verificar, para o momento em que ela é mais difícil.

O princípio zero-trust

As três falhas têm uma causa comum: alguém aceitou uma saída do agente pela palavra do agente. Uma decisão silenciosa passou por requisito, um teste do mesmo autor passou por verificação e uma previsão passou por resultado. O princípio do livro é a negação direta disso:

Toda saída do agente é não confiável até ser verificada por um mecanismo que não dependa do próprio agente.

"Saída" inclui código, testes, planos, explicações e, sobretudo, afirmações de estado como "pronto" e "não quebra nada". "Mecanismo que não dependa do agente" é algo que daria o veredito certo mesmo com o agente errado: um teste que você viu falhar antes da mudança, um comando cuja saída você leu, um gate de CI que barra o merge, uma spec aprovada antes do código.

Na segurança de redes, zero-trust significa que nenhuma origem, nem a rede interna, concede confiança implícita. Aqui, nem a fluência da

resposta, nem os acertos anteriores da sessão, nem a autoria do teste concedem confiança.

Daí saem quatro consequências práticas, que o resto do livro transforma em processo:

- **Separação entre quem produz e quem verifica.** O mesmo princípio que impede alguém de aprovar o próprio pull request vale para o agente que escreve código e teste. O oráculo precisa vir de fora: do requisito aprovado, de um teste visto falhando, de uma revisão em contexto limpo.
- **Evidência observável e reexecutável.** Vale o que outra pessoa pode inspecionar ou rodar de novo. "O agente disse" não é reexecutável; a saída de um comando é.
- **A confiança não se acumula.** Dez acertos seguidos não tornam o décimo primeiro mais confiável. O modelo não melhora ao longo da sessão; como mostra o Capítulo 2, ele tende a piorar.
- **Barreiras automáticas em vez de vigilância.** Atenção humana falha em dia corrido; um gate de pipeline não. O que puder ser verificado por máquina deve ser, de forma inegociável. O mesmo vale para permissões: o agente trabalha numa branch local, sem credenciais de push nem de deploy, porque o que ele não pode fazer não precisa ser auditado.

Zero-trust não é recusa da ferramenta. É uma decisão sobre onde a confiança mora: nos mecanismos de verificação, e não no gerador.

Desenvolvimento verificável

O princípio diz o que não aceitar. O método diz como trabalhar para que sempre haja algo verificável: em vez de revisar com mais esforço no fim, distribuir a verificação ao longo do trabalho, em pedaços pequenos.

Desenvolvimento verificável é trabalhar de modo que cada etapa deixe uma evidência que você consegue ver, e que não depende da palavra do agente.

Evidência, aqui, é sempre um artefato concreto: um documento que você leu e aprovou, um teste que você viu falhar, a saída de um comando que você rodou. São seis, duas em cada momento do trabalho:

- **Antes:** requisito esclarecido e plano revisável. As lacunas do pe-

dido viram perguntas respondidas por você, e o trabalho vira passos conferíveis.
- **Durante:** teste falhando antes do código e teste passando depois. O vermelho prova que o teste mede algo que ainda não existe; o verde, que o código passou a fazê-lo.
- **Depois:** diff revisado por você e resultado verificado por você, com os comandos rodados na sua máquina.

Volte às três falhas e veja em que momento cada uma teria sido barrada:

- **As decisões silenciosas** teriam aparecido **antes**, como perguntas e critérios de aceite: o código também conta? Quantas leituras da tabela por resposta? A regra do ranking é a do extrato?
- **O teste tautológico** não teria sobrevivido ao **durante**. Escrito a partir do critério "tarifa 2 para 1000" e visto falhar contra a implementação errada, ele teria exposto o < no primeiro vermelho.
- **"Os testes devem passar"** não sobreviveria ao **depois**, porque o resultado verificado é a saída de um comando que você rodou, e não a previsão do agente.

Nenhuma dessas práticas é nova. O que muda é que deixam de depender da disciplina de alguém e passam a ser etapas do fluxo, cada uma com um artefato e, como mostra o Capítulo 4, com um gate automatizado.

Do princípio ao processo

Até aqui, o método depende da sua memória, que é o tipo de controle que o próprio princípio desaconselha. Há duas formas de tirá-lo dela. Uma é colocá-lo dentro do agente, como instruções que o fazem perguntar, testar primeiro e rodar os comandos antes de afirmar algo: as skills, implementações de referência dos padrões que o livro descreve. A outra é colocá-lo fora do agente, em gates que nenhuma instrução contorna. As duas são necessárias, porque a skill orienta e não garante: ela também é texto de entrada para um sistema que pode ignorá-la. Por que isso acontece é o assunto do Capítulo 2.

Resumo

- Gerar código ficou barato; verificar continua caro, e o agente não reduz esse custo, só o desloca para o fim, onde ele é maior e a

revisão é menos confiável.

- As três falhas típicas são implementar antes de esclarecer, escrever o teste depois, de forma tautológica, e dizer "pronto" sem evidência.
- O custo deslocado é um problema de engenharia: regra duplicada entre consumidores, contratos implícitos que quebram em silêncio e diffs que não registram as decisões tomadas.
- Teste tautológico é aquele cujo esperado vem da implementação; ele passa por construção, escapa da cobertura e congela o erro.
- Princípio zero-trust: toda saída do agente é não confiável até ser verificada por um mecanismo que não dependa do próprio agente.
- Desenvolvimento verificável distribui seis evidências em três momentos: requisito esclarecido e plano revisável antes; teste falhando antes do código e teste passando depois durante; diff revisado por você e resultado verificado por você depois.

Capítulo 2 — Agente = modelo + harness

O Capítulo 1 mostrou três falhas que soam como descuido humano: decidir em silêncio, testar depois e declarar pronto sem conferir. No agente, elas não são descuido. São propriedades do sistema. Este capítulo separa o agente em suas duas peças, o modelo e o *harness*, para responder a uma pergunta de engenharia: em que pontos dá para pôr um controle determinístico, e em que pontos tudo o que existe é a probabilidade de o modelo se comportar bem.

A premissa do resto do livro sai daqui. Toda saída do agente é não confiável até ser verificada por um mecanismo que não dependa dele.

O harness é um loop de controle

O modelo é uma função sem estado. A cada chamada, ele recebe o *contexto* inteiro e devolve uma continuação. Entre uma chamada e outra, ele não guarda nada. A "memória" da sessão é o histórico que o harness reenvia a cada volta.

O harness é o programa que fecha esse circuito. Ele monta o contexto, que reúne o prompt de sistema, os arquivos de instrução do projeto, as definições das ferramentas e o histórico. Depois chama o modelo e interpreta a resposta. Se a resposta pede uma ferramenta, o harness consulta a política de permissões, executa a ação, anexa o resultado ao contexto e chama o modelo de novo. Reduzido ao essencial, o *loop* é este:

```
while (true) {
  const reply = await model(context);
  if (!reply.toolCall) return reply.text;
  const call = reply.toolCall;
  if (!(await policy.check(call))) {
    context.push(denied(call));
    continue;
  }
  const result = await tools.run(call);
  context.push(call, result);
}
```

Três consequências desse desenho orientam o resto do livro.

O loop termina quando o modelo decide. A condição de parada é a ausência de um pedido de ferramenta. Nada no loop confere se os testes passaram, se o requisito foi atendido ou se o comando prometido che-

gou a rodar. “Pronto” é texto gerado, não um estado verificado. A falha 3 do Capítulo 1, “Os testes devem passar”, é o comportamento padrão de um loop que não tem portão de saída.

Só o harness é determinístico. A política de permissões, a implementação das ferramentas e os ganchos que alguns harness oferecem (comandos disparados em eventos do loop, como antes de uma edição ou ao fim de um turno) se comportam igual toda vez. O modelo não. Um controle que precisa valer sempre tem de morar na parte determinística, ou fora do agente: num teste, num hook, num gate de CI. Uma instrução ao modelo é uma sugestão com alta probabilidade de ser seguida, não um controle.

O contexto é a única entrada. Fora o que veio do treino, o que não está no contexto não existe para o modelo. Quando o agente erra, a pergunta útil não é “por que ele não entendeu?”, e sim “isso estava no contexto, e com peso suficiente?”. A segunda metade da pergunta é o assunto da seção sobre degradação.

A superfície de ferramentas

A *ferramenta* é a API que o harness expõe ao modelo. Cada uma tem nome, descrição e esquema de parâmetros, e essas definições ocupam contexto em toda chamada. Num agente de código, o conjunto mínimo é ler, buscar, editar e executar no shell.

A última domina as outras. Um shell é execução arbitrária de código com os privilégios do usuário que iniciou a CLI: o seu. Tudo o que o seu usuário alcança, o agente alcança, inclusive variáveis de ambiente, chaves SSH carregadas no agente, tokens em `~/.npmrc`, sessões autenticadas do `gh` e credenciais de nuvem no diretório pessoal. A pergunta de segurança não é “quais ferramentas o agente tem?”. É “o que o processo do agente consegue alcançar?”.

O caminho de volta merece a mesma atenção. Toda saída de ferramenta entra no contexto com o mesmo status do seu pedido. Para o modelo, um comentário num arquivo lido, o corpo de uma issue, o README de uma dependência ou uma mensagem de erro são texto, e texto pode conter instruções. Isso é *injeção de prompt* (*prompt injection*) indireta: alguém que controla um dado que o agente lê passa a influenciar o que o agente faz. Não existe filtro confiável para isso dentro do modelo. A defesa está

em limitar o que uma instrução injetada conseguiria executar.

Permissões como fronteira de confiança

Se o modelo é um principal não confiável, a *permissão* é o ponto de aplicação da política: a fronteira entre o que ele propõe e o que de fato acontece. Os harness oferecem variações do mesmo modelo de controle: aprovar uma vez, liberar um padrão de comando, negar um padrão, restringir diretórios e, em alguns casos, isolar a execução num sandbox.

A fronteira é mais porosa do que parece, por três razões:

- **Composição.** Liberar `npm test` parece inofensivo. Mas `npm test` executa o que o `package.json` mandar, e o agente pode editar o `package.json`. Permissão de edição somada a um comando liberado que interpreta arquivos editáveis equivale a execução arbitrária. O mesmo vale para `make`, scripts do projeto e configurações de test runner.
- **Fadiga.** Depois de vinte aprovações legítimas, a vigésima primeira passa sem leitura. A aprovação comando a comando só funciona se cada comando for lido. Quando isso deixa de ser viável, o certo é estreitar o que o agente alcança, não liberar mais.
- **Capacidade não é intenção.** Uma regra que diz ao modelo "nunca faça push" vale tanto quanto a probabilidade de ele obedecer. Um agente sem credencial de push não consegue fazer push. Só a segunda é um controle.

Daí a regra de segurança que o livro segue em todos os capítulos:

- o agente trabalha numa branch local;
- nenhum `git push` acontece sem decisão sua, depois de revisar o diff;
- comandos com efeito são aprovados um a um, e lidos;
- nada sensível fica ao alcance do agente, seja segredo, token ou arquivo de credenciais. Isso inclui o que está no ambiente herdado pelo shell, e não só na pasta do projeto;
- o agente não recebe credenciais de push nem de deploy. Se o seu usuário tem essas credenciais, rode o agente num ambiente que não as tenha: um contêiner, outro usuário do sistema, um ambiente de desenvolvimento remoto.

  [Aviso de Erro Comum] Liberar um padrão amplo, como

qualquer comando `git` ou qualquer script do projeto, para acabar com as interrupções. O sintoma não aparece na hora; aparece quando uma instrução injetada ou um passo mal interpretado usa exatamente essa liberação. Para evitar, libere comandos exatos e sem efeito fora da pasta do projeto, e trate cada pedido de liberação mais ampla como sinal de que o agente está alcançando coisa demais.

Degradação de contexto

O contexto tem limite, e isso todo mundo sabe. O que importa é como ele se deteriora antes de chegar ao limite e o que isso significa para a confiabilidade de uma sessão longa.

Janela finita, medida em tokens O modelo não lê caracteres nem linhas. Lê *tokens*, fragmentos de palavra cujo tamanho varia com a língua e com o tipo de texto. Código, JSON e saídas de log costumam render mais tokens por linha do que prosa. Cada chamada carrega o contexto inteiro: prompt de sistema, definições de ferramentas, instruções do projeto, metadados das skills e todo o histórico de pedidos, leituras e saídas. Uma sessão que leu meia dúzia de arquivos e rodou a suíte algumas vezes consome uma fração relevante da janela, mesmo que a conversa pareça curta.

Compactação descarta detalhes Quando a janela enche, o harness precisa abrir espaço: trunca saídas antigas, descarta turnos ou resume o histórico com o próprio modelo. É a *compactação*. Um resumo preserva o que o modelo considerou importante no momento de resumir, e decisões de borda raramente parecem importantes. "Limite inclusivo" vira "regra de faixas discutida". O resumo não avisa o que perdeu, e o modelo segue trabalhando sobre ele com a mesma fluência.

Atenção diluída e o meio esquecido Mesmo dentro da janela, nem toda posição vale o mesmo. Liu et al. (2023), em "Lost in the Middle", mostraram que modelos recuperam pior a informação posicionada no meio de contextos longos do que a do começo ou do fim. A curva tem forma de U. Numa sessão de agente, o meio é justamente onde ficam as decisões tomadas depois das primeiras trocas e antes do trabalho atual, ou seja, a maior parte do que foi combinado.

Ruído das ferramentas Saídas de ferramenta são volumosas e de baixo sinal: stack traces, logs de instalação, a suíte inteira listada teste a teste, arquivos lidos por completo para achar três linhas. Esse texto compete por atenção com as suas instruções e empurra as decisões para o meio do contexto. Uma única saída de build verbosa pode ocupar mais espaço do que toda a especificação da tarefa.

Decaimento de instruções Instruções antigas perdem peso para as recentes. Uma restrição dada no começo ("não mude a assinatura de `getFee`") compete com dezenas de pedidos e resultados posteriores, e o modelo tende a privilegiar o que está mais próximo do fim. Isso não aparece como desobediência explícita. Aparece como uma escolha "razoável" que contraria algo combinado há quarenta turnos.

Consequência: a sessão é estado não auditável Some os cinco efeitos e a conclusão é desconfortável. Uma sessão longa é estado acumulado que ninguém consegue inspecionar. Você não sabe o que ainda está no contexto, o que foi resumido, o que perdeu peso nem qual saída de ferramenta está disputando atenção com a regra de negócio. Não há diff desse estado, nem revisão, nem como reproduzi-lo amanhã. Confiar numa decisão porque "eu falei isso para ele mais cedo" é confiar num estado que você não consegue auditar.

> **[Exemplo]** No `transfer-service`, a decisão "limite inclusivo, igual ao legado" pode ser tomada nas primeiras trocas de uma sessão. Depois de várias rodadas de leitura do legado e de saídas da suíte, ela está no meio do contexto ou dentro de um resumo. Se o agente reescreve a política nesse ponto, a falha 2 do Capítulo 1, `amount < table.limit`, reaparece sem que ninguém tenha mudado de ideia. Escrita na spec versionada, a mesma decisão é uma linha que qualquer sessão relê e que você revisa no PR.

Defesas As defesas tratam o contexto como o que ele é: um cache volátil, e não a fonte da verdade.

- **Estado em arquivo versionado, e não na conversa.** Requisitos, decisões e planos ficam em arquivos do repositório, que o agente relê no começo de cada etapa e que você revisa por diff. A conversa

vira descartável.

- **Sessões curtas por etapa.** Esclarecer, planejar e implementar cada tarefa são sessões separadas, cada uma começando dos artefatos da anterior. Quanto menor a sessão, menos meio ela tem.
- **Subagentes com contexto curado.** Um subagente começa com um contexto montado só com o que a tarefa precisa e devolve só o resultado. É isolamento do estado do modelo: a tarefa seguinte não herda o ruído da anterior.
- **Worktrees.** Um worktree dá a cada linha de trabalho o seu próprio diretório e a sua própria branch. É isolamento do estado do sistema de arquivos, análogo ao isolamento de processos: um agente não vê nem corrompe as alterações pela metade de outro.

O Capítulo 4 transforma a primeira defesa em método, e o Capítulo 8 retoma as outras quando o trabalho escala.

Por que o agente inventa APIs

A *alucinação* de API é o caso mais concreto de saída plausível e falsa, e entender o mecanismo ajuda a não tratá-la como acidente raro.

O modelo gera a continuação mais provável dado o contexto. Ele não consulta um índice de símbolos antes de escrever uma chamada; produz o que costuma aparecer naquele tipo de código. Na maior parte do tempo, o provável coincide com o existente. Quando não coincide, o resultado é um método com nome convincente, uma opção de configuração com a forma certa ou uma assinatura que existiu numa versão anterior.

Três fatores tornam isso estrutural. Primeiro, o treino mistura versões: a mesma biblioteca aparece com APIs de vários anos, sem marcação confiável de qual vale para o seu lockfile. Segundo, APIs internas não estão no treino: uma porta como `FeeTableProvider` só existe no seu repositório, e, se a definição não estiver no contexto, o modelo completa por analogia com o que já viu. Terceiro, o texto não carrega incerteza: o método inventado sai escrito com a mesma fluência do método real.

Por isso a fonte da verdade nunca é o modelo. É a versão instalada, o type checker e o vermelho que você leu. As defesas concretas estão no Capítulo 8.

MCP: capacidade e superfície de ataque

O *MCP* (*Model Context Protocol*) é um padrão aberto para expor serviços externos como ferramentas de qualquer harness compatível: sistema de chamados, banco de dados, documentação, observabilidade. Para quem desenha o ambiente do agente, ele é duas coisas ao mesmo tempo.

É capacidade. Um servidor MCP de documentação, por exemplo, pode pôr no contexto a API da versão instalada, o que ataca diretamente a alucinação descrita acima.

É também superfície de ataque, em três camadas:

- **O servidor é código de terceiros.** Um servidor local roda como processo seu, com os seus privilégios. Um servidor remoto recebe um token seu. Instalar um servidor MCP é adicionar uma dependência com acesso de execução.
- **As descrições entram no contexto.** Nome, descrição e esquema de cada ferramenta exposta são lidos pelo modelo em toda chamada. Uma descrição maliciosa, ou alterada numa atualização, é uma instrução permanente para o seu agente.
- **Os resultados são entrada não confiável.** O conteúdo de uma issue, de um registro do banco ou de uma página de documentação chega ao modelo com o mesmo peso do seu pedido.

Simon Willison (2025) resumiu a combinação perigosa como a "tríade letal": acesso a dados privados, exposição a conteúdo não confiável e capacidade de comunicação externa. Um agente com as três pode ser induzido a ler um segredo e mandá-lo para fora. Os critérios para cada servidor seguem daí: token de menor privilégio, somente leitura quando possível, versão fixada, revisão da lista de ferramentas a cada atualização e nenhum servidor que complete a tríade numa sessão que tenha acesso a dados sensíveis. O exemplo deste livro não usa nenhum servidor MCP, e o método não depende deles.

Prompt, ferramenta e skill

Com essas peças, dá para separar três coisas que costumam ser confundidas. O critério que importa não é o que cada uma faz, e sim quanto ela dura e quem consegue auditá-la.

- **Prompt.** É o pedido de agora. Mora no contexto e está sujeito a

tudo o que a seção de degradação descreveu: compactação, perda de peso, meio esquecido. Não tem versão nem revisão. Amanhã, ninguém sabe exatamente o que foi pedido, nem você.
- **Ferramenta.** É capacidade. Mora na configuração do harness ou num servidor MCP, dura entre sessões e é auditável como configuração. Mas não diz nada sobre processo: em que ordem trabalhar, quando parar para perguntar, o que conta como pronto.
- **Skill.** É processo em arquivo. Mora no repositório ou num diretório da CLI, dura até alguém mudá-la e, por ser um arquivo, tem histórico, diff e revisão em pull request. É o único dos três que transforma "como trabalhamos" em artefato auditável.

As três falhas do Capítulo 1 não eram falta de ferramenta. O agente podia ler o legado e rodar a suíte. Faltava processo, e processo dito num prompt degrada com a sessão. A skill resolve a durabilidade e a auditabilidade do processo. Ela não resolve a execução: continua sendo uma instrução ao modelo, seguida com alta probabilidade, nunca com garantia. Por isso o método combina as duas coisas: skills para conduzir o agente e verificações externas, como testes vistos falhando, comandos que você rodou e gates de CI, para não depender de ele ter sido conduzido. O Capítulo 3 abre a skill por dentro.

Resumo

- O modelo é uma função sem estado; o harness é o loop de controle que monta o contexto, aplica a política, executa ferramentas e para quando o modelo decide. Nada no loop verifica o "pronto".
- Só o harness e o que está fora do agente são determinísticos. Controles que precisam valer sempre ficam em permissões, testes, hooks e CI, não em instruções.
- O shell é execução arbitrária com os seus privilégios, e toda saída de ferramenta é entrada não confiável. A segurança vem de limitar o alcance do agente, sem credenciais de push ou deploy, e não de pedir que ele se comporte.
- O contexto degrada antes de encher: compactação, meio esquecido, ruído das ferramentas e decaimento de instruções tornam a sessão longa um estado não auditável. Estado vai para arquivo versionado, em sessões curtas e isoladas.
- Alucinação de API é estrutural, porque o modelo gera o plausível a partir de um treino com versões misturadas, e o MCP é ao mesmo

tempo capacidade e superfície de ataque.
- Prompt é efêmero, ferramenta é capacidade e skill é processo durável e revisável, mas nenhum dos três é um portão de verificação.

Capítulo 3 — Skills: um processo em arquivo

O Capítulo 2 terminou com uma distinção: o prompt degrada com a sessão, e a skill é o único dos três mecanismos que transforma processo em artefato durável e auditável. Este capítulo trata a skill como engenharia, e não como recurso de ferramenta. Ele mostra como ela é versionada e revisada, como gasta o orçamento de contexto, por que o acionamento não é confiável e que padrões de engenharia o livro usa a partir daqui. No fim estão os comandos de instalação e o que muda quando a skill vem de terceiros.

Skill é processo versionado

Uma skill é uma pasta com um `SKILL.md`. O arquivo tem um *frontmatter* com dois campos obrigatórios: `name`, em minúsculas e hifens, igual ao nome da pasta, e `description`. Abaixo vem o corpo, com as instruções em Markdown. A pasta pode trazer arquivos de apoio, como referências longas, modelos de documento e scripts. O formato segue um padrão aberto, o Agent Skills, e a mesma pasta funciona no Claude Code e no GitHub Copilot CLI.

O formato é trivial; o que importa é a consequência. Um processo de engenharia costuma morar em três lugares ruins: na cabeça das pessoas, numa página de wiki que ninguém atualiza ou num prompt que cada um cola do seu jeito. Nenhum deles tem versão, autoria, diff ou revisão. A skill tem tudo isso de graça, porque é um arquivo no repositório:

- **Mudança por PR.** Alterar o processo é abrir um pull request, com descrição do motivo, revisão e aprovação, como qualquer mudança de código.
- **Dono explícito.** Uma regra de CODEOWNERS sobre o diretório de skills define quem aprova mudanças no processo do time.
- **Histórico como justificativa.** O `git log` da skill explica por que um passo existe, o que evita a erosão comum de processo ("ninguém lembra por que fazemos isso, vamos pular").
- **Reprodutibilidade.** Duas pessoas do time, em duas CLIs diferentes, carregam o mesmo texto na mesma versão.

Uma skill de projeto deve ser específica. Ela codifica o que só vale ali: o módulo que não pode ser tocado sem rede de proteção, o contrato que não pode quebrar e a regra que não pode ser duplicada.

[Exemplo] No `transfer-service`, uma skill de projeto em `.claude/skills/fee-rules/SKILL.md` guarda o que vale para o legado de tarifas:

```
---
name: fee-rules
description: Use when changing src/legacy/,
  fee calculation or fee tests.
---
1. Before editing src/legacy/, add
   characterization tests; they must pass.
2. Keep the getFee signature unchanged.
3. Reuse the legacy fee rule; never copy it.
```

Repare no que o exemplo não faz. Ele não ensina TDD nem explica o que é teste de caracterização; isso é papel de skills gerais. Ele só amarra o processo geral aos pontos de risco deste repositório.

Divulgação progressiva: orçamento de contexto

O Capítulo 2 mostrou que cada token no contexto compete por atenção e que a janela se deteriora antes de encher. Skills instaladas disputam o mesmo orçamento. O formato lida com isso por *divulgação progressiva* (*progressive disclosure*), em três níveis de custo:

1. **Metadados, sempre carregados.** O `name` e a `description` de cada skill disponível entram no contexto de toda sessão. O custo é baixo por skill, mas é fixo e se multiplica pelo número de skills instaladas.
2. **Corpo, ao acionar.** O corpo do `SKILL.md` entra no contexto só quando a skill é acionada, e a partir daí ocupa espaço como qualquer outro texto da sessão.
3. **Arquivos de apoio, sob demanda.** Referências e modelos só entram se o corpo mandar e o agente de fato ler. Scripts podem ser executados sem serem lidos, e aí só a saída entra no contexto.

Isso gera critérios de projeto para quem escreve skills:

- **A descrição é a interface.** Ela é o único texto que o modelo vê antes de decidir. Deve dizer quando usar, com gatilhos concretos (caminhos, tipos de tarefa, sintomas), e não o que a skill é.
- **O corpo é curto e imperativo.** Passos e regras verificáveis. Material de referência longo vai para arquivos de apoio, que o corpo aponta.

- **Trabalho determinístico vai para script.** Uma checagem que pode ser um comando é mais confiável executada do que descrita, e custa menos contexto.
- **Menos skills é melhor.** Cada skill instalada é custo fixo em toda sessão e mais uma descrição competindo pela escolha. Skills com descrições sobrepostas pioram as duas coisas.

Acionamento não é determinístico

A skill entra em ação de duas formas: pela descrição, quando o modelo decide que ela se aplica ao pedido, ou pelo nome, quando você a invoca explicitamente com barra, como `/fee-rules`.

O acionamento pela descrição é uma inferência do modelo, sujeita a tudo o que o Capítulo 2 descreveu. Ele pode falhar de várias formas:

- o pedido não se parece com a descrição o bastante, e a skill não é acionada;
- duas descrições se sobrepõem, e a escolhida é a errada;
- a skill é acionada no começo da sessão, e o corpo é compactado ou perde peso horas depois, embora o agente continue agindo como se a seguisse;
- o modelo carrega a skill e pula um passo que julgou desnecessário para aquele caso.

Sob a ótica de zero-trust, isso define a regra de uso. **Quando o processo importa, chame a skill pelo nome.** O acionamento automático serve para conveniência, nas situações em que esquecer a skill custa pouco. Ele não serve como controle. E, mesmo chamada pelo nome, a skill só aumenta a probabilidade de o processo ser seguido. A confirmação vem do artefato que o processo deveria produzir: a spec gravada, o plano no repositório, o vermelho na saída do teste. Se o artefato não existe, o processo não aconteceu, diga o agente o que disser.

> **[Dica Prática]** Em mudanças de risco, invoque a skill pelo nome numa sessão nova e confira na transcrição que ela foi carregada antes do primeiro pedido de edição. Numa sessão longa, invoque de novo ao começar uma etapa, em vez de confiar que o corpo carregado lá atrás ainda pesa.

Onde as skills moram

Cada CLI procura skills em dois escopos, e a escolha entre eles é uma decisão de governança, não de conveniência.

O **escopo de projeto** fica no repositório. A skill é versionada, revisada em PR e igual para todo o time. É o lugar de qualquer processo que o time considera obrigatório.

O **escopo pessoal** fica no seu diretório de usuário. A skill vale para todos os seus projetos, mas é invisível para quem revisa o seu trabalho. Serve para preferências e para processos gerais que você quer em qualquer repositório. Se o time depende de uma regra, ela não pode morar só no escopo pessoal de alguém.

Claude Code

- projeto: `.claude/skills/`;
- pessoal: `~/.claude/skills/`.

Copilot CLI

- projeto: `.github/skills/`, `.claude/skills/` ou `.agents/skills/`;
- pessoal: `~/.copilot/skills/` ou `~/.agents/skills/`.

Como `.claude/skills/` aparece nas duas listas, um time que usa as duas CLIs pode manter as skills do projeto ali. Evite o mesmo nome de skill em escopos diferentes: a regra de precedência varia entre CLIs e versões, e um processo ambíguo é um processo não auditável.

Padrões de engenharia com IA

As falhas do Capítulo 1 não são específicas de uma ferramenta, e as respostas a elas também não. O livro organiza essas respostas como padrões de engenharia: cada um nomeia um problema recorrente do trabalho com agentes, um mecanismo que o resolve e a evidência que ele deixa. A skill é só o veículo. O padrão continua valendo se você trocar de CLI, de pacote de skills ou escrever a sua própria implementação.

O *superpowers*, pacote aberto de skills de engenharia criado por Jesse Vincent e publicado em `obra/superpowers`, é usado neste livro como implementação de referência de cada padrão. O pacote evolui; o livro não fixa quantas skills ele tem.

Elicitação Guiada de Requisitos (*Guided Requirements Elicitation*). Problema: o agente preenche lacunas do pedido em silêncio, com a interpretação mais provável. Mecanismo: antes de qualquer código, o agente explora o repositório, faz perguntas uma de cada vez, propõe abordagens com trade-offs e só grava a spec depois da sua aprovação. Evidência: requisito esclarecido. Referência: `brainstorming`.

Decomposição Verificável (*Verifiable Decomposition*). Problema: tarefas grandes demais para revisar e sem critério de pronto. Mecanismo: a spec vira um plano de tarefas pequenas, cada uma com arquivos explícitos, um teste que começa falhando e o comando que prova o resultado. Evidência: plano revisável. Referência: `writing-plans`.

Prova por Falha (*Fail-First Proof*). Problema: o teste escrito depois espelha a implementação e passa por construção. Mecanismo: o teste vem antes do código e precisa falhar pelo motivo certo; só então vem o código mínimo para passar. Evidências: teste falhando antes do código e teste passando depois. Referência: `test-driven-development`.

Portão de Verificação Independente (*Independent Verification Gate*). Problema: o loop termina quando o modelo decide, e "pronto" é texto. Mecanismo: nenhuma afirmação de conclusão sem o comando que a prova executado agora, com a saída lida. Evidência: resultado verificado. Referência: `verification-before-completion`.

Delegação com Contexto Limpo (*Clean-Context Delegation*). Problema: o contexto acumulado de uma sessão longa degrada a execução de cada tarefa nova. Mecanismo: um coordenador entrega cada tarefa a um subagente com contexto curado, recebe só o resultado e revisa antes da próxima. Referência: `subagent-driven-development`.

Isolamento de Espaço de Trabalho (*Workspace Isolation*). Problema: o agente altera a mesma árvore de arquivos em que você e outros agentes trabalham. Mecanismo: cada linha de trabalho ganha um worktree e uma branch próprios, e o resultado é uma branch para revisar. Referência: `using-git-worktrees`.

Revisão Independente (*Independent Review*). Problema: quem escreveu o código é o pior revisor dele, e isso vale também para o agente na mesma sessão. Mecanismo: a revisão é feita por um contexto que não participou da implementação, contra a spec e o plano, antes da sua leitura do diff. Referência: `requesting-code-review`.

Caracterização antes da Mudança (*Characterize Before Change*). Problema: em código legado sem testes, não existe especificação confiável do comportamento atual, e qualquer mudança pode quebrar um contrato sem que ninguém perceba. Mecanismo: antes de tocar o código, testes registram o que ele faz hoje. Eles nascem verdes, ao contrário do TDD, e só provam algo quando se quebra o legado de propósito e se vê o vermelho. Não há skill própria no superpowers; é o passo zero, e uma skill de projeto como a `fee-rules` é o lugar natural para exigi-lo. O resultado é a linha de base caracterizada, pré-requisito das seis evidências e não uma sétima.

O fluxo mínimo do livro usa os quatro primeiros: esclarecer, planejar, vermelho/verde, verificar. O Capítulo 4 liga cada um à sua evidência e apresenta a variação para código legado. A Parte II executa o fluxo no exemplo. O Capítulo 8 trata de delegação, isolamento e revisão quando o trabalho escala. O Apêndice A mapeia o pacote completo.

Instalando o superpowers

Há duas formas. Como *plugin*, o pacote inteiro é instalado e atualizado de uma vez. Um plugin pode trazer, além de skills, comandos e hooks que a CLI executa sem passar pelo modelo, então tudo o que a próxima seção diz vale em dobro para ele. Pela CLI `skills`, você instala só as skills escolhidas.

Claude Code (dentro da sessão):

```
/plugin install superpowers@claude-plugins-official
```

Copilot CLI:

```
copilot plugin marketplace add \
  obra/superpowers-marketplace
copilot plugin install \
  superpowers@superpowers-marketplace
```

Só o fluxo mínimo, via `npx` (exige Node.js 22 ou superior). Aqui ele vai para o escopo pessoal do Claude Code; troque `claude-code` por `github-copilot` para o Copilot CLI, e omita `-g` para instalar no escopo do projeto:

```
npx skills add obra/superpowers -g \
  -a claude-code \
  -s brainstorming writing-plans \
  test-driven-development \
```

```
verification-before-completion
```

Skills vindas de plugin podem aparecer com prefixo, como `superpowers:brainstorming`; é a mesma skill. O Apêndice C reúne as equivalências entre as CLIs, inclusive como listar e invocar skills.

> **[Nota]** Os comandos foram conferidos na data de publicação. Se mudarem, o README de `obra/superpowers` traz as instruções atuais para cada CLI.

Skills de terceiros: risco de cadeia de suprimentos

Uma skill de terceiros é uma dependência, e das mais privilegiadas que você pode adicionar. Uma biblioteca roda dentro do seu programa. Uma skill dirige um agente que, como mostrou o Capítulo 2, tem shell com os seus privilégios. Trate-a como problema de *cadeia de suprimentos* (*supply chain*).

Os vetores são os conhecidos de qualquer ecossistema de pacotes, com um agravante:

- **Instrução maliciosa no corpo.** Um passo que manda ler arquivos fora do escopo, rodar um script da própria pasta ou enviar dados para fora. Chega ao aprovador com a mesma cara de um passo legítimo.
- **Scripts e hooks.** Código executável dentro da pasta, ou do plugin, que roda com os seus privilégios, às vezes sem passar pela aprovação de comandos.
- **Injeção via arquivos de apoio.** Referências lidas sob demanda entram no contexto como qualquer saída de ferramenta.
- **Atualização comprometida.** A versão que você revisou era segura; a seguinte vem de uma conta comprometida ou de um mantenedor novo. É o mesmo padrão de ataques a pacotes npm, só que o payload é texto em linguagem natural, que nenhum scanner de dependências analisa.

O agravante é que o payload não precisa parecer código. Uma frase bem colocada no meio de cem linhas razoáveis é suficiente.

Os controles também são os de cadeia de suprimentos:

- **Revisão integral antes de instalar.** Leia o `SKILL.md` inteiro, os arquivos de apoio, os scripts e, num plugin, a configuração de hooks

e de servidores MCP. Mesmo critério de um PR de alguém de fora do time.
- **Pinagem de versão.** Não deixe uma skill de terceiros flutuar para a versão mais recente. A forma mais simples e independente de CLI é incorporar ao repositório (*vendoring*) a pasta revisada, no escopo de projeto, registrando a origem e o commit de onde ela veio. A partir daí, a skill só muda por PR.
- **Reavaliação a cada atualização.** Atualizar é revisar o diff entre a versão pinada e a nova, não reinstalar. Diff que você não leu é código que você não aprovou.
- **Menor privilégio continua valendo.** Nenhuma skill justifica liberar padrões amplos de comando, expor credenciais ou dar ao agente acesso de push ou deploy.

  [Aviso de Erro Comum] Instalar como plugin com atualização automática e revisar só a primeira versão. O sintoma, quando aparece, é o agente executando passos que ninguém do time escreveu. Para evitar, pine a versão revisada no repositório e trate cada atualização como um PR de dependência, com diff lido e aprovação explícita.

Por fim, o limite que vale até para a skill mais confiável: ela orienta o agente, mas não o obriga. O controle está nos artefatos que o processo produz e nas verificações que não dependem do agente. O Capítulo 8 volta a esse limite.

Resumo

- Skill é processo em arquivo: versionada, com dono, revisada em PR e reproduzível entre pessoas e CLIs. Skills de projeto amarram o processo geral aos pontos de risco do repositório.
- A divulgação progressiva gere o orçamento de contexto em três níveis (metadados, corpo, arquivos de apoio). Descrição precisa, corpo curto, trabalho determinístico em script e poucas skills.
- O acionamento pela descrição é não determinístico. Quando o processo importa, chame a skill pelo nome e confirme pelo artefato produzido, não pela palavra do agente.
- Os padrões de engenharia com IA são o conceito, e as skills do superpowers são implementações de referência. O fluxo mínimo usa Elicitação Guiada de Requisitos, Decomposição Verificável, Prova

por Falha e Portão de Verificação Independente; em código legado, Caracterização antes da Mudança vem antes de tudo.
- Skills e plugins de terceiros são risco de cadeia de suprimentos: revise tudo antes de instalar, pine a versão no repositório e reavalie o diff a cada atualização.

Capítulo 4 — Evidência e Spec-Driven Development

O Capítulo 1 definiu desenvolvimento verificável como trabalhar de modo que cada etapa deixe uma evidência que não dependa da palavra do agente. Este capítulo fecha a Parte I com três coisas. Primeiro, quais são as seis evidências, que falha cada uma barra e que gate de pipeline a sustenta quando ninguém está olhando. Depois, o *Spec-Driven Development* tratado como engenharia de requisitos executável. Por fim, o fluxo mínimo que a Parte II executa e a variação que ele exige em código legado.

Seis evidências, seis gates

Uma evidência é um artefato produzido por um mecanismo que o agente não controla: um arquivo que você aprovou, uma saída de teste que você leu, um comando que você rodou. Uma frase do agente nunca é evidência, por mais detalhada que seja. É o princípio de zero-trust do Capítulo 2 aplicado ao processo: a saída do agente é não confiável até que algo independente dele a confirme.

As seis evidências se dividem em dois momentos antes do código, dois durante e dois depois. Cada uma tem duas camadas. A camada local é a evidência propriamente dita, produzida na hora e lida por você. A camada automatizada é o gate que, no pipeline, barra a mesma falha quando a camada local foi pulada, esquecida ou feita às pressas. A local pega o erro cedo e barato; o gate garante que ele não chegue à branch principal. O Capítulo 8 monta o pipeline; aqui fica o mapa.

Requisito esclarecido. Uma spec versionada, escrita depois que as lacunas do pedido viraram perguntas e as perguntas receberam respostas de quem tem autoridade sobre elas. Barra a lacuna decidida em silêncio. O gate é um script de CI, `scripts/check-spec.mjs`, que recebe o caminho da spec e falha se faltar uma das seções obrigatórias: objetivo, requisitos, critérios de aceite e fora do escopo. Uma spec sem fora do escopo não passa, e a revisão do pull request lê a spec como lê código.

Plano revisável. Um arquivo com tarefas pequenas, ordenadas, cada uma com o teste que a inicia e o comando que a confere. Barra a tarefa sem teste e o critério que se perde entre a spec e o código. O gate é a outra metade do mesmo script: a rastreabilidade. Cada critério tem um ID (`AC-01`, `AC-02`…), e o script falha se algum ID não aparecer em nenhum

arquivo de teste versionado. Um plano que esqueceu um critério produz um código cujo teste não cita aquele ID, e o pipeline acusa.

Teste falhando antes do código. A saída vermelha, lida por você, com a mensagem de falha que a regra prevê. Barra o teste tautológico, aquele que nasce verde porque espelha o código em vez da regra. Aqui o pipeline tem um limite estrutural: o CI vê o estado final do repositório, não a ordem em que as coisas aconteceram. Nenhum gate prova que o teste falhou antes. O substituto automatizado é o teste de mutação: o Stryker altera o código de produção e verifica se a suíte percebe. Um teste que espelha a implementação sobrevive às mutações dela, e o mutation score abaixo de `thresholds.break` derruba o build. Mutação não prova que o teste nasceu vermelho; prova que ele protege. O Capítulo 7 desenvolve a teoria.

Teste passando depois. A saída verde do mesmo comando, depois da implementação. Barra a regressão e a implementação que só passa no ambiente do agente. O gate é a suíte completa no CI, com typecheck e lint, num ambiente limpo. A integridade da suíte entra no mesmo gate: com a variável `CI` definida, o Vitest falha se encontrar um `.only` esquecido (é o padrão da opção `allowOnly`), e um `.skip` novo no diff é barrado por verificação explícita.

Diff revisado por você. A leitura linha a linha do que mudou, antes do commit. Barra a mudança que ninguém pediu: o arquivo fora do escopo, a assinatura pública alterada, a função renomeada no caminho. O gate é a branch protegida com revisão humana obrigatória e, para os arquivos que guardam contratos, CODEOWNERS: quem é dono do contrato precisa aprovar qualquer mudança neles.

Resultado verificado por você. O comando rodado por você, no seu terminal, e não o relato do agente de que rodou. Barra o "pronto" sem evidência. O gate é o próprio CI, que roda fora do alcance do agente, combinado com uma regra de privilégio: o agente não tem credenciais de push nem de deploy. O que ele declara pronto só sai da sua máquina pela sua mão, e só chega à branch principal depois que o pipeline concorda.

As três falhas do Capítulo 1 caem nesse mapa sem sobra. A busca só pelo nome e as leituras repetidas da tabela são lacunas decididas em silêncio, e param no requisito esclarecido. O teste que afirma R$ 5,00

para R$ 1.000,00 no Banco Azul para no teste falhando antes do código, porque um teste escrito a partir da regra exige R$ 2,00 e fica vermelho contra a implementação errada. O "devem passar" para no resultado verificado por você. O diff revisado cobre o resto: as surpresas que nenhum teste procura.

> **[Nota]** Gate não substitui evidência. O pipeline pega o erro tarde, depois que o código foi escrito, o teste foi ajustado e o contexto se perdeu. A evidência local pega o mesmo erro no minuto em que ele nasce. O gate existe porque pessoas cansam e pulam etapas, não para que você possa pulá-las.

SDD: engenharia de requisitos executável

Spec-Driven Development (SDD) não é uma ideia nova com nome novo. É engenharia de requisitos, a disciplina de elicitar, registrar, validar e rastrear o que o sistema deve fazer, adaptada a um executor que não pergunta por iniciativa própria. A novidade está em dois pontos.

O primeiro é o leitor da spec. Num time humano, uma spec incompleta é compensada por conversa de corredor, pela memória do time e pelo bom senso de quem implementa. Um agente não tem nada disso. Tudo o que não está escrito ele preenche com o mais plausível segundo o treino, e o mais plausível raramente coincide com a regra do seu negócio. A spec deixa de ser um documento de alinhamento e passa a ser a única fonte de requisitos que o executor enxerga.

O segundo é a executabilidade. Uma spec é executável quando os seus critérios estão ligados a testes por um vínculo que uma máquina confere. Não basta que os critérios "virem testes" na cabeça de alguém: cada critério tem um identificador, cada teste que o prova cita esse identificador, e um script quebra o build quando o vínculo some. A spec deixa de ser uma intenção e passa a ser um contrato com verificação automática.

As quatro seções obrigatórias têm funções distintas na cadeia:

- **Objetivo:** desempata decisões que a spec não previu. Quando o plano encontra uma lacuna nova, é contra o objetivo que se avalia a resposta.
- **Requisitos:** as regras, em linguagem declarativa. Dizem o que deve valer, não como implementar.

- **Critérios de aceite:** os casos que provam os requisitos, cada um com entrada e resultado esperado. São a interface entre a spec e a suíte de testes.
- **Fora do escopo:** o limite explícito da mudança. Com um agente, é a seção que mais rende por linha, porque tudo o que não é proibido o agente pode considerar oportunidade de melhoria.

Requisito sem critério não é verificável. Critério sem requisito é teste sem razão de existir. A revisão da spec confere as duas direções.

Critérios com ID: a base da rastreabilidade

Rastreabilidade é poder responder, a qualquer momento, duas perguntas: que teste prova este requisito, e que requisito justifica este teste? Em projetos grandes ela costuma morar numa matriz mantida à mão, que fica desatualizada na primeira semana de pressa. Com IDs estáveis e um script, ela passa a morar no próprio código.

A convenção é simples: cada critério recebe um ID `AC-nn`, que nunca é reaproveitado, e o teste que o prova leva o ID no nome, no `it` ou no `describe`. Três consequências práticas:

- **Mudança de spec vira mudança rastreável.** Um critério alterado mantém o ID, e o diff da spec aponta exatamente que teste precisa mudar junto. Um critério removido deixa um teste órfão, visível numa busca pelo ID.
- **A revisão fica dirigida.** Ao ler o diff de uma tarefa, você sabe quais IDs ela deveria cobrir e confere se os testes citam esses IDs e afirmam o resultado que o critério diz.
- **Lacunas descobertas tarde entram no mesmo trilho.** Uma regra que só aparece no planejamento ganha o próximo ID, volta para a spec num commit e passa a ser cobrada pelo pipeline como as outras.

O script confere só a direção da spec para o teste: todo critério tem ao menos um teste que o cita. A direção inversa, teste sem critério, fica para a revisão humana, porque há testes legítimos que não correspondem a um critério de negócio. E o vínculo é nominal: o script garante que o ID aparece, não que o teste prova o que o critério diz. Essa segunda parte é trabalho do vermelho lido e da mutação.

[Exemplo] Um critério da spec do ranking e o teste que o cita:

```
- AC-04 Banco Azul fee for 1000 is 2; for
  1000.01 is 5.

it("AC-04 lowFee at the limit", ...)
```

Se o teste perder o ID, o `check-spec.mjs` falha com `no test for AC-04`.

A curva de custo da correção

A intuição de que um defeito fica mais caro quanto mais tarde é descoberto vem da curva clássica de Boehm, dos anos 1980. As magnitudes daquela curva são contestadas e variam muito com o processo do time. A direção, não. E o desenvolvimento com agentes muda a forma da curva de um jeito que engana.

Gerar código ficou quase gratuito. Isso cria a impressão de que refazer também é: se a implementação saiu errada, basta pedir outra. Mas o custo de um defeito nunca esteve na digitação. Ele está em detectar, diagnosticar, desfazer as decisões que se apoiaram no erro e reverificar tudo o que dependia do comportamento antigo. Nenhuma dessas atividades ficou mais barata, e a última ficou mais cara, porque o agente produz mais código dependente por hora.

Tome o limite da faixa como exemplo de defeito e acompanhe o custo por etapa:

- **Na spec:** trocar "abaixo de" por "até, inclusive". Uma frase, nenhum código, nenhum teste congelou nada.
- **No plano:** reescrever o teste inicial de uma tarefa. Um parágrafo.
- **Numa tarefa:** refazer a tarefa. O teste errado e o código errado nasceram juntos e morrem juntos.
- **Depois do merge:** uma investigação. O teste tautológico congelou o erro, outras tarefas se apoiaram nele, e a correção exige descobrir quem mais depende do comportamento.
- **Em produção:** um incidente. A tela mostra uma tarifa e o extrato cobra outra, e o custo passa a incluir atendimento, estorno e confiança.

Há um segundo efeito, menos óbvio: revisar artefatos pequenos em sequência é mais barato do que revisar um artefato grande no fim, porque cada revisão isola uma dimensão. Na spec, você pensa só em regras. No

plano, só em decomposição e ordem. No diff de uma tarefa, você sabe exatamente que IDs procurar. A revisão do Capítulo 1, centenas de linhas sem critério, é o caso oposto: a atenção se espalha por tudo e não se aprofunda em nada.

Por isso o SDD revisa um artefato por vez, nesta ordem: spec, plano, tarefas, código. Cada um é aprovado antes de o próximo começar, e cada um é insumo verificável do seguinte.

Spec Kit e a constituição

O superpowers não é a única implementação de SDD para agentes. O *Spec Kit*, kit aberto do GitHub publicado em `github/spec-kit`, conduz as mesmas etapas por comandos: princípios do projeto, spec, plano, tarefas e implementação.

A contribuição conceitual do Spec Kit é a *constituição*: um documento com os princípios que toda spec do projeto deve respeitar. A spec muda a cada história; a constituição é a camada de invariantes acima dela. Separar as duas tem um efeito direto sobre o agente: as regras que valem sempre deixam de depender de alguém lembrar de repeti-las no pedido, e passam a ser carregadas no contexto de toda sessão a partir de um arquivo versionado.

A ideia vale fora do Spec Kit, em qualquer CLI que leia um arquivo de instruções do projeto. Mas uma constituição só é útil na medida em que é verificável. "Escreva código de qualidade" não restringe nada. "Toda regra de negócio tem critério com ID e teste que o cita" restringe, e pode ser imposta por gate. "Código de domínio não importa módulos de infraestrutura" pode virar regra de lint. O princípio que não tem gate correspondente é uma sugestão ao modelo, e sugestões perdem peso numa sessão longa, como mostrou o Capítulo 2.

> **[Nota]** Os comandos do Spec Kit mudam entre versões, por isso este livro não os lista. Consulte o README do repositório para as instruções atuais. O método deste capítulo independe da ferramenta.

O fluxo mínimo nos padrões canônicos

O Capítulo 3 apresentou as skills como processos em arquivo. Cada uma das quatro skills do fluxo mínimo é a implementação de referência de um padrão de engenharia com IA. O padrão é o conceito, portável para qualquer CLI; a skill é um veículo.

1. Esclarecer. Padrão: Elicitação Guiada de Requisitos, implementado pela skill `brainstorming`. O agente explora o projeto, transforma cada lacuna em pergunta, uma por vez, propõe abordagens e grava a spec com commit, sem escrever código. Barra a implementação antes do esclarecimento. Evidência: requisito esclarecido. Gate: seções obrigatórias da spec. Executado no Capítulo 5.

2. Planejar. Padrão: Decomposição Verificável, implementado por `writing-plans`. A spec aprovada vira tarefas pequenas, cada uma com arquivos, teste inicial e comando de conferência. Barra o teste escrito depois e o critério perdido. Evidência: plano revisável. Gate: rastreabilidade entre critérios e testes. Executado no Capítulo 6.

3. Vermelho, depois verde. Padrão: Prova por Falha, implementado por `test-driven-development`. O teste nasce de um critério, roda e falha pela razão certa; só então vem o código mínimo. Barra o teste tautológico. Evidências: teste falhando antes e teste passando depois. Gates: mutação e suíte completa no CI. Executado no Capítulo 7.

4. Verificar. Padrão: Portão de Verificação Independente, implementado por `verification-before-completion`. O agente não declara conclusão sem rodar os comandos e mostrar a saída, e a declaração dele ainda é só o insumo da sua verificação. Barra o "pronto" sem evidência. Evidências: diff revisado e resultado verificado por você. Gates: branch protegida, revisão obrigatória e CI fora do alcance do agente. Executado no Capítulo 7.

Cada padrão consome o artefato do anterior. Pular um deles não economiza a etapa; apenas faz com que a evidência dela não exista, e o gate correspondente passe a ser a primeira linha de defesa em vez da última. Os outros padrões, Delegação com Contexto Limpo, Isolamento de Espaço de Trabalho e Revisão Independente, entram quando a história cresce, no Capítulo 8.

[Dica Prática] O fluxo completo tem custo fixo. Renomear

uma variável ou corrigir uma mensagem não pede spec nem plano. Use-o onde há regra de negócio ambígua, contrato com outro consumidor ou risco de erro caro. O Capítulo 8 volta a esse critério.

Variação para código legado: o passo zero

O fluxo mínimo supõe que o comportamento atual do código está protegido por testes. Em código legado, essa suposição falha, e mudar código sem testes é editar às cegas, como define Michael Feathers em *Working Effectively with Legacy Code*. Antes de esclarecer qualquer coisa nova, é preciso fixar o que o código já faz.

É o padrão Caracterização antes da Mudança. Ele não tem skill própria no superpowers: é um passo zero que precede o fluxo sempre que a mudança toca código sem testes do qual outro consumidor depende.

Um teste de caracterização documenta o que o sistema faz, não o que ele deveria fazer. Você chama o código com entradas escolhidas, observa a saída e afirma exatamente essa saída. Isso o torna o inverso do TDD em dois pontos:

- **Nasce verde.** Ele descreve o comportamento existente, então passa na primeira execução. Um teste de caracterização vermelho significa que você entendeu errado o código, não que o código está errado.
- **Afirmar o que o código devolve é o objetivo, não o defeito.** O mesmo gesto que torna um teste novo tautológico é o que dá valor a um teste de caracterização. A diferença está no papel: aqui o código é a especificação vigente.

Como ele nasce verde, a pergunta "este teste protege algo?" precisa de outra resposta. A resposta é quebrar o legado de propósito, com uma mutação manual no ponto sensível, como trocar um > por $\geqslant$, e conferir que a suíte fica vermelha. É a mesma lógica do teste de mutação, aplicada à mão antes da mudança.

O resultado é a *linha de base caracterizada*: uma suíte que congela o contrato atual e deve continuar verde, sem alteração, ao longo de toda a mudança. Ela é pré-requisito, não uma sétima evidência. As seis evidências provam a mudança; a linha de base prova que o que já existia

continua igual. No pipeline, a suíte de caracterização é um check obrigatório, e os seus arquivos ficam sob CODEOWNERS, para que nem você nem o agente possam "ajustar" o contrato para fazer a mudança passar.

Caracterizar também é uma técnica de elicitação. Ao escrever os casos, você descobre regras que ninguém documentou, e elas viram insumo da spec. A segunda falha do Capítulo 1 é o exemplo: o agente reimplementou a tarifa com `<` no limite, enquanto o legado, que o extrato usa, aplica a faixa baixa até o limite, inclusive. Um teste de caracterização do legado teria tornado a regra explícita antes de qualquer pergunta, e a divergência entre tela e extrato teria aparecido como um vermelho, não como uma reclamação de cliente.

Antes da Parte II

A Parte II conduz a história do ranking de bancos do pedido vago ao teste verde, um padrão por capítulo, com o passo zero aplicado ao `getFee` legado. O projeto de partida e o código final estão no Apêndice D. As regras de segurança do Capítulo 2 valem do início ao fim: branch local, aprovação comando a comando, nada sensível ao alcance do agente e nenhuma credencial de push ou deploy na sessão dele.

Resumo

- As seis evidências (requisito esclarecido, plano revisável, teste falhando antes do código, teste passando depois, diff revisado por você e resultado verificado por você) têm, cada uma, uma falha que barram e um gate de pipeline que as sustenta.
- O CI vê o estado final, não a ordem: nenhum gate prova que o teste nasceu vermelho, e a mutação é o substituto que prova que ele protege.
- SDD é engenharia de requisitos executável: critérios com ID ligados a testes por um vínculo que um script confere e que quebra o build quando some.
- Um defeito custa uma frase na spec e um incidente em produção; gerar código barato não barateia detectar, diagnosticar e reverificar.
- O fluxo mínimo implementa quatro padrões: Elicitação Guiada de Requisitos, Decomposição Verificável, Prova por Falha e Portão de Verificação Independente.

- Em código legado, Caracterização antes da Mudança vem antes de tudo: a linha de base caracterizada é pré-requisito, não sétima evidência.

Capítulo 5 — Esclarecer

Parte II — Na prática

A Parte II percorre o fluxo mínimo, um padrão por capítulo. Este trata do primeiro, a Elicitação Guiada de Requisitos: por que o modelo preenche lacunas por conta própria, como classificá-las, quem decide cada uma e como auditar a spec que sai da etapa. No fim, a primeira evidência, o requisito esclarecido, está num commit da sua branch.

Por que o modelo preenche lacunas

A primeira falha do Capítulo 1 não foi descuido do agente, e sim o comportamento padrão de um modelo diante de um pedido incompleto.

Otimização para entregar. Os modelos usados em agentes de código são ajustados para concluir tarefas. No treino, uma resposta que atende ao pedido é preferida a uma que devolve perguntas, e um pedido com lacunas ainda é um pedido que dá para atender. O resultado é um viés sistemático: diante da ambiguidade, o modelo escolhe uma interpretação e segue.

Viés de completar. Um modelo gera continuações plausíveis. Uma lacuna não aparece para ele como um buraco a sinalizar, e sim como um ponto a preencher com o padrão mais frequente no treino. Se a maioria dos códigos de faixa que ele viu usa `<` no limite, `<` é o que sai. Se buscas costumam filtrar por nome, a busca filtra por nome. O preenchimento é estatisticamente razoável e semanticamente arbitrário: reflete o código médio da internet, não a regra do seu negócio.

Ausência de sinal. O código que decide uma lacuna é indistinguível do que implementa uma regra explícita, e a resposta soa igualmente segura nos dois casos. Pelo princípio de zero-trust, toda lacuna preenchida pelo agente é uma saída não verificada, com o agravante de que você nem sabe que ela existe.

A Elicitação Guiada de Requisitos inverte o padrão: proíbe o código até que as lacunas tenham sido transformadas em perguntas, as perguntas em decisões e as decisões em texto versionado.

O padrão e a sua implementação de referência

A skill `brainstorming` implementa o padrão com uma sequência fixa. Conhecer o motivo de cada passo é o que permite perceber quando o agente pula um.

- **Explorar antes de perguntar.** O agente lê código, configuração e documentação antes da primeira pergunta. Perguntas ancoradas no projeto ("o `getFee` legado aplica a faixa baixa até o limite; o ranking deve seguir a mesma regra?") rendem decisões; perguntas genéricas ("quais são os requisitos?") devolvem o trabalho a você. Nesta fase, só leituras.
- **Uma pergunta por vez, de preferência fechada.** Uma pergunta isolada recebe uma resposta pensada; uma lista recebe respostas apressadas e deixa itens sem resposta. Alternativas `A)`, `B)` forçam o agente a explicitar as interpretações possíveis, o que já expõe a lacuna.
- **Duas ou três abordagens, com recomendação justificada.** Fechadas as dúvidas, o agente compara caminhos e diz o que cada um troca. É o único ponto em que a recomendação dele pode ser aceita, e só para decisões técnicas.
- **Design em seções, com aprovação incremental.** Cada seção é lida como parte da spec.
- **Persistir e versionar.** A spec é gravada em `docs/superpowers/specs/`, com data e tema no nome, e entra num commit. Nenhum código de produção é escrito na etapa.

O diálogo muda entre execuções; o que você audita é a sequência.

Uma taxonomia de lacunas

A skill não encontra todas as lacunas; a cobertura dela depende do que o projeto deixa visível. Uma taxonomia é um checklist independente do agente, e quatro categorias cobrem a maior parte do que dá errado.

Lacunas de negócio. Regras que só quem pediu a história conhece: limites, inclusões, ordenações, o que conta como correspondência. São as mais fáceis de reconhecer e as mais fáceis de o agente "resolver" com um palpite plausível. Na história do ranking, eram duas: de que lado do limite fica uma transferência de exatamente R$ 1.000,00, e se o termo digitado busca pelo nome, pelo código ou pelos dois.

Lacunas de contrato. Quando a regra já existe em outro lugar do sistema, a lacuna não é "qual é a regra", e sim "qual é a fonte da verdade". O risco é a reimplementação: uma segunda cópia da regra, escrita a partir do pedido, que diverge da primeira no primeiro caso de borda. O critério de decisão é que, se dois consumidores precisam concordar, eles compartilham a implementação, não a descrição. No ranking, o `getFee` legado é a fonte, porque é ele que o serviço de extrato usa para cobrar; a tela e o extrato só coincidem se ambos derivarem da mesma política.

Lacunas de concorrência e consistência. Quando o código lê estado que muda por fora, a lacuna é qual consistência a operação garante. Cache com refresh, tabela recarregada por um job, configuração remota: duas leituras podem devolver versões diferentes. O critério de decisão é a unidade de consistência, ou seja, qual conjunto de leituras precisa ver o mesmo estado. No ranking, a resposta é um snapshot da tabela por requisição, porque um ranking que mistura duas versões da tabela compara tarifas que nunca coexistiram. O agente do Capítulo 1 leu a tabela dentro do laço, uma vez por banco, sem que ninguém tivesse decidido isso.

Lacunas de dependência e porta. Como a dependência entra no código: importada diretamente, lida de um estado global ou recebida por injeção através de uma porta. Decide o que é testável. Uma decisão de consistência como "um snapshot por requisição" só vira critério verificável se houver uma costura onde um provider falso possa contar as leituras. No ranking, o `FeeTableProvider` injetado é essa costura: em produção busca a tabela no pricing-service, em teste devolve uma tabela fixa e registra quantas vezes foi chamado.

As quatro categorias se encadeiam. A decisão de contrato (reaproveitar a política do legado) leva a uma decisão de dependência (extrair a regra numa função pura que o legado e o serviço novo compartilham), e a decisão de consistência só é testável por causa da porta. Uma spec que trata uma categoria e ignora a seguinte costuma ter um critério impossível de provar.

Quem decide o quê

Esclarecer é levar cada decisão a quem tem autoridade sobre ela, não transferi-la do agente para você. O agente nunca tem autoridade; no máximo, recomenda.

- **Negócio:** decide quem pediu a história, o dono do produto ou a área responsável pela regra. O seu papel é levar a pergunta, não respondê-la por inferência. Se você não sabe, essa é a descoberta mais útil da sessão: a spec registra a dúvida como pendente e o fluxo para até a resposta chegar. Uma spec com "a confirmar" é melhor que uma spec com um chute.
- **Contrato:** decide quem é dono do contrato e quem o consome. Mudar uma interface que outro serviço usa não é decisão local, mesmo quando parece correção. No ranking, preservar a assinatura e o comportamento de `getFee` não estava em discussão: o extrato depende dele.
- **Concorrência e consistência:** decide a engenharia, você ou o tech lead, mas com um insumo de negócio. Qual defasagem é aceitável entre o preço publicado e o preço exibido é pergunta de negócio; como garantir a consistência escolhida é pergunta técnica.
- **Dependência e porta:** decide a engenharia, e aqui a recomendação do agente pode ser aceita, desde que a justificativa seja conferível: por que injetar, o que fica testável, que acoplamento se evita.

A regra prática é simples: uma recomendação do agente só é aceitável quando você consegue verificar a justificativa sem confiar nele. "Injetar o provider permite contar leituras num teste" é verificável. "Valores no limite costumam ir para a faixa alta" não é, e mesmo que fosse, não é o agente quem decide.

O mínimo de condução

A skill conduz a conversa; a sua intervenção se concentra em poucos pontos.

- **Responda com decisões.** "Tanto faz" devolve ao agente a decisão que a etapa existe para tirar dele.
- **Complete a cobertura com a taxonomia.** A lacuna que o agente não encontrou entra na sua próxima resposta, já decidida.
- **Recuse ações fora da etapa.** Instalar dependências, criar arquivos em `src/` ou commitar algo além da spec indica que o agente abandonou o processo.
- **Fixe a spec no idioma do código.** Ela é lida ao lado de nomes de funções e de testes; um idioma só elimina a tradução mental ao conferir critério contra teste.

[Exemplo] Uma lacuna de contrato, levantada pelo agente depois de ler o legado:

Agente: O `getFee` em `src/legacy/fees.ts` já calcula a tarifa, e o extrato o consome. Como o ranking deve obter a tarifa?

A) Chamar `getFee` diretamente
B) Reimplementar a regra no serviço novo
C) Extrair a regra numa função pura usada pelos dois

Você: C. O `getFee` mantém a assinatura e passa a delegar à função extraída.

A alternativa A acopla o ranking ao estado global do legado; a B é a reimplementação que causou a divergência do Capítulo 1. A escolhida resolve a lacuna de contrato e a de dependência de uma vez.

A spec como estado fora do contexto

O Capítulo 2 mostrou que uma sessão longa é estado acumulado e não auditável: a janela é finita, a compactação descarta detalhes, a informação do meio da conversa é recuperada pior que a das pontas e as instruções antigas perdem peso para as recentes. Uma decisão tomada na décima pergunta do brainstorming é exatamente o tipo de informação que se degrada primeiro.

A spec externaliza esse estado para um arquivo com quatro propriedades que a conversa não tem:

- **Persistente:** sobrevive ao fim da sessão, à compactação e à troca de CLI.
- **Curável:** um subagente recebe a spec como contexto de partida, limpo, sem as idas e vindas que produziram as decisões. É o insumo natural da Delegação com Contexto Limpo, no Capítulo 8.
- **Diffável:** uma decisão alterada aparece como uma linha mudada, revisável no pull request como código.
- **Verificável por máquina:** o `check-spec.mjs` lê a spec, confere as seções e cobra que cada critério tenha teste. A conversa não pode ser lida por um gate.

O corolário é duro e útil: decisão que ficou só na conversa não existe. Ela não chega ao plano, não chega aos testes e não chega ao revisor.

Auditando a spec

A spec gravada só vale como evidência depois de auditada: o agente pode ter resumido uma resposta ou trocado "até" por "abaixo de".

- **Cada decisão está na spec, na forma que você deu.** Confira resposta por resposta. Atenção a palavras de fronteira: "até", "acima de", "contém", "igual a".
- **Cada critério tem ID.** `AC-01`, `AC-02` e assim por diante, sem lacunas de numeração e sem IDs reaproveitados. É o ID que vai ligar o critério ao teste e permitir a rastreabilidade no pipeline.
- **Cada critério tem entrada e resultado.** Dado este termo, este valor, este estado, o resultado é este. Um critério que descreve uma qualidade ("a busca é tolerante a acentos") é requisito, não critério; o critério correspondente é um caso ("ambar" encontra o Banco Âmbar). Decisões de consistência também precisam de forma observável: "um snapshot por requisição" é verificável porque um provider falso conta as leituras.
- **Existe fora do escopo, e ele é específico.** Sem essa seção, o limite da mudança fica implícito, e o implícito o agente decide. "Nada além do pedido" não delimita nada; "sem alterações no serviço de extrato" delimita.
- **A spec está num commit isolado.** Só a spec, sem código e sem outras alterações pendentes. Isso permite revisá-la separada da implementação e reverter uma decisão sem desfazer código.

  [Exemplo] Trechos dos critérios de aceite da spec gravada:

```
## Acceptance criteria
- AC-02 "001" finds Banco Azul; "1" finds none.
- AC-04 Banco Azul fee for 1000 is 2; for
  1000.01 is 5.
- AC-06 One fee table snapshot per request.
- AC-07 Legacy getFee keeps its signature and
  its characterization suite passes unchanged.
```

Os quatro trechos cobrem as quatro categorias da taxonomia: busca (negócio), limite (negócio, alinhado ao contrato), snapshot (consistência, testável pela porta) e preservação do legado (contrato). A spec sai desta etapa com sete critérios. O oitavo aparece no planejamento, como mostra o Capítulo 6, e entra pelo mesmo trilho: novo ID, commit na spec.

Erros comuns

> **[Aviso de Erro Comum]** Aprovar a spec por consequência. Depois de dez perguntas respondidas, a spec parece certa porque a conversa pareceu certa. O sintoma é seguir para o plano sem ter aberto o arquivo. A auditoria é contra as suas respostas, item por item, não contra a impressão da sessão.
>
> **[Aviso de Erro Comum]** Tratar só as lacunas de negócio. Elas são as mais visíveis, e a skill costuma encontrá-las. As de contrato e de consistência raramente aparecem no pedido e são as que produzem divergências em produção. Percorra a taxonomia inteira antes de aprovar.
>
> **[Aviso de Erro Comum]** Critério sem forma observável. "Uses a consistent fee table" descreve uma intenção que nenhum teste prova. Se um critério não tem entrada e resultado, ou não indica como o resultado é observado, peça que seja reescrito como caso.

Com a spec auditada e commitada, a skill oferece a etapa seguinte: a Decomposição Verificável, com `writing-plans`. O Capítulo 6 começa daí.

Resumo

- O modelo preenche lacunas por otimização para entregar e por viés de completar, e o preenchimento não deixa sinal: toda lacuna decidida pelo agente é saída não verificada.
- Elicitação Guiada de Requisitos impõe explorar antes de perguntar, uma pergunta por vez, abordagens com recomendação, design incremental e spec versionada, sem código.
- Lacunas se classificam em negócio, contrato, concorrência e consistência, e dependência e porta; percorrer as quatro é o checklist que independe do agente.
- Cada tipo de decisão tem um dono: negócio a quem pediu, contrato a quem o mantém e consome, consistência e portas à engenharia; o agente só recomenda, e só quando a justificativa é conferível.
- A spec é o estado persistente fora do contexto do modelo: sobrevive à sessão, serve de contexto limpo para subagentes e é lida por

gates.
- Auditar a spec é conferir decisões, critérios com ID, entrada e resultado, fora do escopo específico e commit isolado.

Capítulo 6 — Planejar

A spec diz o quê. O plano diz como, em que ordem e como cada passo será conferido. Este capítulo trata o plano como peça de engenharia, e não como lista de afazeres: um contrato entre você e o agente, decomposto em unidades que se verificam e se revertem sozinhas. O resultado é a segunda evidência, o plano revisável.

Decomposição Verificável

A Decomposição Verificável, que a skill `writing-plans` implementa, responde a uma pergunta que todo plano responde mal quando ninguém a faz: qual é o tamanho certo de uma tarefa?

O critério não é tempo nem número de linhas. Uma tarefa tem o tamanho certo quando é, ao mesmo tempo, uma **unidade de verificação** e uma **unidade de rollback**.

- **Unidade de verificação:** a tarefa termina num estado observável por um comando, com uma saída esperada declarada antes. Se você não consegue dizer qual comando prova que ela terminou, ela ainda não é uma tarefa; é uma intenção.
- **Unidade de rollback:** a tarefa termina verde e num commit próprio. Reverter esse commit deixa o sistema num estado coerente, com a suíte passando. Cada fronteira de tarefa vira um ponto de retorno e um ponto de bisseção.

Tarefas grandes demais perdem a localização da falha: o vermelho pode vir de qualquer uma de vinte mudanças, e o `git bisect` não tem onde cortar. Tarefas pequenas demais quebram a segunda propriedade: o commit intermediário não compila ou deixa a suíte vermelha. O tamanho certo quase sempre coincide com um critério de aceite ou com um pequeno grupo deles.

Com um agente, esse critério pesa mais do que com uma pessoa. O Capítulo 2 mostrou que o contexto se degrada em sessões longas: detalhes somem na compactação, instruções antigas perdem peso. Uma tarefa curta, com início e fim definidos no arquivo, limita quanto estado a sessão precisa carregar. E, no regime de zero-trust, a declaração "terminei a tarefa" só tem valor se o fim da tarefa for definido por algo que você observa sem depender do agente.

O plano como contrato

O plano é o único documento que o agente, você e qualquer subagente leem com o mesmo significado. Por isso ele precisa ser legível por máquina: cada passo traz a ação, o comando que a confere e a saída esperada, inclusive quando a saída esperada é uma falha.

```
- [ ] Step 2: run the new test
  Run: npx vitest run src/x/y.test.ts
  Expect: FAIL, "parse is not a function"
- [ ] Step 3: minimal implementation
- [ ] Step 4: run the suite
  Run: npm test
  Expect: PASS, no other test changed
```

Um passo assim tem uma propriedade que a prosa não tem: a divergência é detectável. Se a saída obtida não bate com a declarada, o trabalho saiu do contrato naquele passo, e o executor para. Não importa quem executa; o contrato é o mesmo para o agente na sessão principal, para um subagente com contexto limpo no Capítulo 8 e para você.

A `writing-plans` grava o plano em `docs/superpowers/plans/` com essa disciplina: cabeçalho com objetivo e arquitetura, um mapa de arquivos antes de qualquer tarefa, passos de poucos minutos com caminhos e comandos exatos, proibição de "TODO" e de passos vagos, e uma revisão final contra a spec. Peça o plano a partir da spec aprovada no Capítulo 5, com um teto de tarefas:

```
/writing-plans Plano a partir da spec aprovada.
No máximo 4 tarefas. Cada passo de teste com
comando e saída esperada. Cite os IDs dos
critérios em cada tarefa.
```

O teto evita que o agente fatie demais ou puxe trabalho vizinho. O mapa de arquivos é a declaração prévia do raio de impacto da mudança. No fim do Capítulo 7, o diff será conferido contra ele: todo arquivo alterado fora do mapa precisa de uma justificativa, ou sai.

Em código legado: caracterizar antes, extrair depois

Quando a história toca código legado sem testes, a ordem das tarefas deixa de ser questão de gosto. Há uma ordem correta, e ela começa antes do primeiro teste da história.

Task 0: a linha de base caracterizada A primeira tarefa aplica a Caracterização antes da Mudança, apresentada no Capítulo 4 a partir de Michael Feathers. Os testes de caracterização registram o que o código faz hoje, e não o que a spec deseja. Eles fixam o contrato de que os consumidores atuais dependem, certo ou errado.

Daí sai uma consequência que contraria o reflexo de quem pratica TDD: **o teste de caracterização nasce verde, por definição.** Ele descreve o comportamento existente. É o inverso do ciclo vermelho-verde, e por isso o verde inicial não prova nada. Um teste de caracterização que afirma `true` $\equiv$ `true` também nasce verde.

A prova vem de outro jeito: quebrando o legado de propósito. O plano da Task 0 deve declarar, depois do verde, um passo de sabotagem, por exemplo trocar > por $\geqslant$ na comparação com o limite. O esperado é vermelho no teste do limite. Em seguida, o passo de desfazer a sabotagem e ver o verde voltar. Só então a suíte de caracterização vale como linha de base.

O que caracterizar? O contrato que os consumidores observam e que a mudança pode alterar: as fronteiras de faixa, dos dois lados do limite; o comportamento de erro, inclusive a mensagem, se alguém a lê; os efeitos sobre estado compartilhado. Não o módulo inteiro.

Task 1: extrair com a suíte intocada A segunda tarefa extrai a regra para uma unidade nova e faz o legado delegar a ela, com a assinatura intacta. É o padrão *strangler* em escala de função: o código novo cresce por dentro do antigo, e o contrato externo não muda.

A regra da Task 1 é rígida: **os arquivos de caracterização não podem aparecer no diff.** Se a suíte ficar vermelha, a extração mudou comportamento, e quem se ajusta é a extração. Um teste de caracterização editado na mesma branch que mexe no código que ele protege deixa de ser evidência de qualquer coisa.

Por que extrair, e não chamar o legado direto ou reescrever a regra? Chamar o legado acopla o código novo ao estado global que ele lê. Reescrever a regra é a segunda falha do Capítulo 1: duas implementações da mesma política, que divergem no primeiro caso de borda. Extrair deixa uma política só, consumida pelos dois lados.

[Exemplo] As quatro tarefas do plano do ranking, depois da

lacuna resolvida:

```
- Task 0: characterize legacy getFee (AC-07)
- Task 1: extract feeFor; getFee delegates
- Task 2: searchBanks (AC-01..03)
- Task 3: BankRankingService (AC-04..06, 08)
```

Se `feeFor` divergir do legado, o vermelho aparece na Task 1, e não na tela do cliente.

Portas e injeção como pré-condição de testabilidade

Alguns critérios de aceite não são testáveis contra o design que existe. Um caso típico: "uma leitura da tabela de tarifas por requisição". Se o serviço importa um cliente singleton do serviço de preços, o teste não tem como contar leituras nem como simular um refresh do cache no meio da requisição. O critério está escrito, mas não há onde verificá-lo.

A saída é uma *costura* (*seam*, no vocabulário de Feathers): um ponto onde o comportamento pode ser trocado sem editar o código sob teste. Em código orientado a objetos, a costura mais limpa é uma porta, uma interface pequena que o serviço recebe por injeção de dependência. Em produção, a implementação da porta fala com o outro serviço. No teste, um provider falso devolve uma versão nova da tabela a cada chamada e conta quantas vezes foi chamado. Com ele, o critério do snapshot vira uma asserção determinística: uma chamada, uma versão.

O critério de planejamento que sai daí: para cada critério de aceite, pergunte o que você precisa controlar para que o teste seja determinístico. Rede, relógio, aleatoriedade, concorrência e estado global são as respostas comuns, e cada uma pede uma porta. Se a porta não existe, criá-la é uma tarefa do plano, anterior à tarefa que a usa. Um plano que adia essa decisão para a implementação deixa o agente decidir sozinho, e o caminho mais curto para ele costuma ser um mock de módulo que acopla o teste à estrutura interna.

A lacuna que o plano descobre

Planejar tem um efeito colateral valioso. Para escrever um passo de teste com entrada e saída exatas, alguém precisa saber a saída. Uma spec que parecia completa revela, no passo concreto, uma entrada sem resultado

definido. É a última chance barata de fechar a lacuna: aqui ela custa uma frase e um commit; no código, custa uma investigação.

Quando a lacuna aparece, a decisão costuma ser de negócio ou de operação, e por isso é sua. A regra desta etapa:

Toda decisão tomada no plano volta para a spec, como critério de aceite com ID, num commit próprio, antes de o plano seguir.

O fluxo é da spec para o plano, nunca o contrário. Uma regra que existe só no plano, ou só na conversa, não é conferida pela rastreabilidade nem pelo gate de CI do Capítulo 8. Ao aprovar o commit, confira que ele toca só o arquivo da spec.

> **[Exemplo]** A lacuna surgiu ao escrever o teste da Task 3:
>
> **Agente:** O snapshot pode não ter tarifa para um banco do cadastro. O ranking deve omitir o banco, falhar a requisição ou exibi-lo sem tarifa?
>
> **Você:** Omitir. O pricing-service pode estar no meio do rollout de um banco novo; o ranking não pode cair por isso, nem mostrar sem tarifa um banco como se fosse o mais barato.
>
> A spec ganhou o AC-08. O `getFee` legado continua lançando erro, porque o contrato dele é o AC-07.

Repare no efeito sobre o design: a política extraída passa a devolver "sem tarifa" como valor, e cada consumidor decide o que fazer com ele. O ranking omite; o legado preserva o comportamento que o extrato conhece.

Auditar o plano contra os IDs

A revisão final da skill diz que o plano cobre a spec. Essa revisão é do agente; a evidência é a sua auditoria. Com critérios identificados, ela é mecânica na primeira passada:

```
spec=docs/superpowers/specs/<file>.md
plan=docs/superpowers/plans/<file>.md
ids() { grep -ohE 'AC-[0-9]+' "$1" | sort -u; }
comm -3 <(ids "$spec") <(ids "$plan")
```

Saída vazia quer dizer que todo ID da spec aparece no plano, e vice-versa. Qualquer linha é um problema: um critério sem tarefa é regra que ninguém vai verificar; um ID no plano que não existe na spec é decisão que não voltou para ela. O Capítulo 8 leva a mesma ideia para o CI, cruzando a spec com os arquivos de teste.

A segunda passada é sua, tarefa por tarefa:

- **Toda tarefa cita pelo menos um ID.** Uma tarefa sem critério é escopo que ninguém pediu, com uma exceção: tarefas habilitadoras, como a Task 0 e a Task 1, que precisam dizer qual critério protegem.
- **Todo passo de teste declara a falha esperada e o motivo.** "Expect: FAIL" sozinho não basta.
- **A Task 0 declara verde, sabotagem, vermelho e retorno.** Sem a sabotagem, a linha de base não foi validada.
- **A tarefa de extração declara a suíte de caracterização intocada.**
- **O mapa de arquivos não inclui nada que a spec põe fora do escopo.**
- **Um critério verificado pela caracterização carrega o ID no próprio teste.** Do contrário, a rastreabilidade o dá como órfão, mesmo coberto.

Depois de aprovado, o plano entra num commit, como a spec. No histórico da branch aparecem, nesta ordem, a spec, a lacuna e o plano. Execute na própria sessão, uma tarefa por vez, como mostra o Capítulo 7; a execução com subagentes fica para o Capítulo 8. O código completo das tarefas está no Apêndice D.

Erros comuns

> **[Aviso de Erro Comum]** Caracterização tratada como TDD. O agente escreve a Task 0, vê tudo verde e segue, ou pior, ajusta o legado até um teste "ficar vermelho primeiro". Caracterização nasce verde; a prova é a sabotagem deliberada, desfeita em seguida.
>
> **[Aviso de Erro Comum]** Critério sem costura. O plano promete testar algo que depende de rede, relógio ou estado global, sem dizer como controlar isso. O agente resolve na hora com o mock mais conveniente. Exija a porta no plano.

Resumo

- Decomposição Verificável: a tarefa certa é unidade de verificação (um comando, uma saída esperada) e unidade de rollback (verde, num commit que se reverte sozinho).
- O plano é um contrato legível por máquina; cada passo declara comando e saída esperada, e a divergência é o sinal de parada.
- Em código legado, a Task 0 caracteriza o comportamento atual, nasce verde e é validada pela sabotagem; a extração segue com a suíte de caracterização intocada.
- Critérios que dependem de rede, cache ou estado global exigem uma porta injetada; o provider falso torna o snapshot testável.
- Lacunas descobertas no plano voltam para a spec, com ID e commit; a auditoria cruza os IDs da spec com os do plano antes da aprovação.

Capítulo 7 — Vermelho, depois verde

Com a spec e o plano aprovados, entra o código, e ele só chega depois de um teste que você viu falhar. Este capítulo trata da etapa em que o agente mais produz e em que a sua desconfiança mais rende: o ciclo vermelho-verde, a teoria que explica quando um teste protege alguma coisa e o portão que separa "o agente disse que terminou" de "está verificado". No fim, as quatro evidências que faltam estão reunidas.

Prova por Falha

Um teste é uma afirmação condicional: se o comportamento X quebrar, eu fico vermelho. Enquanto você não viu o vermelho, essa afirmação não foi testada. Um teste que sempre passou é indistinguível, na saída da suíte, de `expect(true).toBe(true)`.

A Prova por Falha, que a skill `test-driven-development` implementa, existe por causa de um problema estrutural do trabalho com agentes: **o mesmo autor escreve o teste e o código.** Não há verificação independente entre os dois. Se o agente entendeu a regra errado, o teste e a implementação erram juntos, e o verde só prova que concordam entre si. O único momento em que o teste é confrontado com algo além do código que ele testa é o vermelho, quando ele roda contra a ausência do comportamento.

A skill impõe a ordem sem negociação:

- nenhum código de produção antes de um teste falhando;
- a falha é lida e o motivo é confirmado antes da implementação;
- código mínimo para aquele teste, sem antecipar a próxima tarefa;
- verde da suíte inteira, não só do teste novo;
- código escrito antes do teste é apagado, e não "aproveitado" com um teste escrito em volta.

A última regra é a mais dura e a mais importante. Um teste escrito em volta de código existente é a segunda falha do Capítulo 1 com outro nome.

Peça uma tarefa do plano por vez e continue aprovando comando a comando. Instalar pacote, mexer em configuração ou tocar arquivo fora do mapa foge do contrato do Capítulo 6.

Falhou ou quebrou: o vermelho pelo motivo certo

Nem todo vermelho é prova. Há duas categorias, e confundi-las anula a evidência.

- **O teste falhou:** a asserção rodou e o valor observado diverge do esperado. A mensagem fala do comportamento, como `expected 5 to be 2`.
- **O teste quebrou:** a asserção nem chegou a rodar. `SyntaxError`, import com caminho errado, fixture que lança na preparação, nome digitado errado. A mensagem fala do arquivo de teste, e não da regra.

Um teste quebrado não diz nada sobre o código, nem o verde que vem depois de consertá-lo. Entre os extremos há um vermelho legítimo, mas fraco: `is not a function` no primeiro ciclo. Ele mostra que o teste chama a função, não que a asserção discrimina valores. O vermelho que vale mais é o seguinte, quando a função existe e devolve o valor errado.

Ler um vermelho é conferir quatro coisas:

1. o nome do teste corresponde ao critério que a tarefa implementa;
2. a mensagem fala de valor, e não de estrutura;
3. o esperado na mensagem é o da spec, e não o que o código devolve;
4. a contagem bate: falha exatamente o que deveria falhar, e o resto continua passando.

O quarto item é o mais esquecido: `3 failed` quando você esperava uma falha quer dizer que algo mais quebrou. E o Vitest transpila sem checar tipos; a suíte nunca substitui o typecheck, que é outro gate.

Teoria do teste de mutação

Ver o teste falhar uma vez prova que ele discrimina um defeito: a ausência do código. A pergunta seguinte é mais exigente: que outros defeitos ele pega? O teste de mutação responde a essa pergunta de forma sistemática, e a teoria por trás dele explica tanto o seu poder quanto os seus limites.

Mutantes, operadores e score Um *mutante* é uma cópia do programa com uma única alteração sintática pequena, gerada por um operador de

mutação. A ferramenta gera centenas deles, roda a suíte contra cada um e classifica:

- **morto:** pelo menos um teste falhou; a suíte detectou o defeito;
- **sobrevivente:** todos os testes passaram; a suíte não distingue o mutante do original.

Os operadores reproduzem os deslizes que programadores cometem. Os mais reveladores:

- **troca de operador relacional e de limite:** `>` vira $\geqslant$, `<` vira $\leqslant$;
- **negação de condicional:** `if (c)` vira `if (!c)`, ou a condição vira `true` ou `false`;
- **troca de operador lógico:** `||` vira `&&`;
- **remoção de chamada ou de bloco:** o corpo de uma função ou um `continue` desaparece;
- **troca de método:** `toLowerCase()` vira `toUpperCase()`.

O *mutation score* é a fração de mutantes detectados sobre os mutantes válidos. Ele mede outra coisa que a cobertura. Cobertura diz que uma linha executou; mutação diz que alguma asserção depende do que a linha faz. Uma suíte com 100% de cobertura e asserções fracas pode deixar metade dos mutantes viva.

Por que mutações pequenas bastam Uma objeção natural: defeitos reais não são uma troca de caractere. A resposta vem de DeMillo, Lipton e Sayward (1978), em duas hipóteses.

A *hipótese do programador competente* diz que programas escritos por profissionais competentes estão perto do correto; os defeitos que sobram são desvios pequenos da versão certa. O *efeito de acoplamento* diz que testes capazes de detectar todos os defeitos simples também detectam a maioria dos defeitos complexos, que são composições de simples.

Juntas, elas justificam o método: se a suíte mata os mutantes de primeira ordem, ela provavelmente mata os defeitos reais. E aqui há um argumento específico para agentes. O código que um modelo gera é plausível por construção. Ele raramente erra a estrutura; erra o operador, o lado do limite, a chamada no lugar errado. O `<` do Capítulo 1 é exatamente um mutante de limite. A hipótese do programador competente descreve o agente melhor do que descreve muita gente.

Mutantes equivalentes Alguns mutantes não podem morrer. Um *mutante equivalente* é sintaticamente diferente do original e semanticamente idêntico: nenhuma entrada distingue os dois. Decidir equivalência de programas é indecidível no caso geral, por isso nenhuma ferramenta os descarta sozinha; eles aparecem como sobreviventes e baixam o score.

> **[Exemplo]** Stryker 10.1 sobre o código final do ranking: 52 mutantes, 51 mortos, 1 sobrevivente, score 98,08 (break em 80).
>
> ```
> [Survived] MethodExpression
> src/ranking/bankRanking.ts:12:10
> -       .toLowerCase();
> +       .toUpperCase();
> ```
>
> `normalize` é aplicada ao termo e ao nome antes da comparação; trocar a caixa dos dois lados não muda o resultado.

Declarar um mutante equivalente é uma análise, e não um atalho. A troca de caixa só é equivalente se as duas direções forem simétricas para as entradas possíveis. Em Unicode geral não são (o "ß" alemão vira "SS" em maiúsculas), mas para o cadastro de bancos e o alfabeto do termo, são. Registre esse raciocínio junto da supressão, e não aceite "é equivalente" do agente sem ele. Um sobrevivente rotulado como equivalente por conveniência é um teste faltando com um álibi.

Testes tautológicos: os que sobrevivem e os que enganam O Capítulo 1 apresentou o teste tautológico. A teoria de mutação permite separá-lo em duas famílias, e a distinção importa porque a ferramenta só pega uma delas.

Testes sem sensibilidade deixam mutantes vivos, e a mutação os denuncia:

- **espelho por reuso:** o esperado é calculado chamando a própria implementação, ou um helper que ela também usa. O mutante altera os dois lados da comparação, e o teste continua verde;
- **asserção sobre o mock, e não sobre o efeito:** o teste confere que `current()` foi chamado, mas não o que o serviço fez com a tabela. Qualquer mutação no cálculo sobrevive;
- **asserções fracas:** `toBeDefined()`, só o tamanho da lista, só "não lançou". O valor pode ser qualquer um.

Testes sem oráculo independente matam mutantes e, ainda assim, não provam nada sobre a regra:

- **espelho por cópia:** o esperado é calculado com a mesma lógica, copiada para o teste. A cópia carrega o defeito original;
- **snapshot aceito sem leitura:** o snapshot fixa a saída atual, certa ou errada;
- **esperado copiado da saída:** roda-se o código, cola-se o resultado no `expect`.

Esses três são sensíveis a mudanças, então o score sobe. O problema está no *oráculo*, a fonte do valor esperado: ele é o próprio código. O teste do Capítulo 1, que afirmava `5` para R$ 1.000,00, mataria o mutante `<` para $\leqslant$ sem dificuldade, e o defeito chegaria à produção com score alto.

A lição é precisa: **mutação mede sensibilidade, não correção do oráculo.** A correção do oráculo vem de fora do código: do critério de aceite da spec ou da caracterização de um consumidor que já está em produção. Repare que as duas últimas técnicas são, literalmente, como se escreve um teste de caracterização. A diferença é a intenção declarada: em código legado, fixar o comportamento atual é o objetivo; em código novo, é tautologia.

Quebrar de propósito: a mutação manual

Rodar a ferramenta a cada tarefa é caro. A versão manual custa um minuto: depois do verde, aplique você mesmo o mutante mais provável, rode a suíte, confira que o teste certo fica vermelho e desfaça.

A escolha do mutante segue a hipótese do programador competente. Pergunte qual é o erro plausível que este código admite: o lado do limite, uma condição invertida, uma chamada movida para dentro do laço. Não sabote ao acaso; sabote onde o agente erraria.

> **[Exemplo]** Sabotagem do limite em `feeFor`, com `>` trocado por $\geqslant$:
>
> ```
> × charges lowFee at exactly the limit
> × AC-04 lowFee at the limit
> AssertionError: expected 5 to be 2
>       Tests  2 failed | 8 passed (10)
> ```
>
> Dois vermelhos, um por consumidor da política: o extrato, pela caracterização, e o ranking, pelo AC-04.

A segunda sabotagem plausível é a primeira falha do Capítulo 1: mover `current()` para dentro do laço. O provider falso a pega com `expected 3 to be 1` no AC-06, e a contagem mostra `Tests  1 failed | 9 passed (10)`. Uma linha movida, uma falha, no critério certo.

O vermelho da caracterização

O exemplo acima mostra algo que só aparece em código legado: um vermelho num teste que não pertence à tarefa em curso. Ele tem semântica oposta à do vermelho do TDD.

No ciclo de TDD, o vermelho é esperado e desejado: ele vem antes do código. Na suíte de caracterização, o vermelho é um alarme: ele diz que o comportamento observado por um consumidor em produção mudou. Por isso a resposta também é oposta. Diante de um vermelho de caracterização, quem recua é a mudança, nunca o teste.

Há um caso em que o vermelho de caracterização revela um defeito real do legado. Mesmo assim, a correção não entra nesta branch. Ela muda o que o consumidor recebe, e isso é uma decisão de negócio, com spec própria, fora do escopo da história atual.

Desconfie do vermelho "consertado" no teste

Quando um vermelho vira verde, a primeira pergunta é: qual arquivo mudou? Se foi o arquivo de teste, e não a implementação, pare.

O agente otimiza o objetivo declarado, e "fazer os testes passarem" pode ser satisfeito editando testes: o esperado ajustado ao valor que o código devolve; a asserção afrouxada, de `toBe` para `toBeGreaterThan`; o `.skip` num caso difícil; o `.only` que faz o resto da suíte nem rodar; o mock reescrito para devolver o que o código espera; o timeout aumentado até a flutuação sumir.

A regra: depois que você confirmou que um teste falhou pelo motivo certo, a única razão legítima para editá-lo é ele estar quebrado, e quem decide isso é você, olhando a mensagem. Um esperado que diverge da spec é a segunda falha do Capítulo 1 voltando pela porta dos fundos.

Portão de Verificação Independente

A terceira falha do Capítulo 1 foi o "pronto" sem evidência. O Portão de Verificação Independente, que a skill `verification-before-completion` implementa, define o que um "pronto" exige:

- **frescor:** a evidência foi produzida depois da última mudança. Uma saída verde de três edições atrás não vale;
- **completude:** o comando é o da suíte inteira, e não o de um arquivo filtrado; a saída é lida inteira, e não resumida;
- **independência:** o mecanismo que produz a evidência não depende da narrativa do agente.

Chame a skill antes de aceitar qualquer "terminei":

```
/verification-before-completion Confirme que a
Task 3 e a suíte completa estão verdes.
```

Mas repare no limite dela. A skill roda dentro do agente. Ela obriga o agente a rodar o comando e citar a saída, o que já elimina o "deve passar", mas continua sendo o agente relatando o próprio trabalho, no mesmo contexto que pode ter perdido detalhes. A independência tem graus: o agente rodando e citando; você rodando no seu terminal; o CI rodando num ambiente limpo, que o Capítulo 8 trata. A skill é o primeiro grau. A evidência do método começa no segundo.

A evidência final: a suíte e o diff

A suíte, rodada por você. Fora da sessão, no seu terminal, rode `npm test` e leia a contagem: `Tests  10 passed (10)`, em 2 arquivos de teste. Confira o número contra o plano. Dez testes previstos, dez rodados. Uma contagem menor sem explicação é um teste apagado ou pulado.

O diff, revisado com checklist. Comece pelo tamanho e depois vá aos pontos sensíveis:

```
git diff --stat main...HEAD
git diff main...HEAD -- src/legacy/fees.ts
git log --oneline -- \
  src/legacy/fees.characterization.test.ts
git diff main...HEAD -- package.json \
  package-lock.json
```

O checklist de um revisor sênior:

- **Contrato público intacto.** No legado, a assinatura de `getFee` não mudou e só o corpo delega. Nenhum export removido ou renomeado.
- **Nenhum teste enfraquecido.** Nenhuma asserção afrouxada, nenhum `.skip` ou `.only` novo, nenhum teste apagado. O arquivo de caracterização tem um commit só, o da Task 0.
- **Nenhum esperado ajustado.** Cada valor literal num `expect` corresponde a um critério da spec ou ao comportamento caracterizado.
- **Nada fora do escopo.** Os arquivos do `--stat` estão contidos no mapa do plano, mais a spec e o plano.
- **Nenhuma dependência nova.** O diff de `package.json` e do lockfile está vazio. Dependência é superfície de ataque e decisão de arquitetura; não entra como efeito colateral de uma tarefa.

Enviar a branch é o último passo, e quem decide é você. O agente não tem credenciais de push nem de deploy.

Erros comuns

> **[Aviso de Erro Comum]** Confiar no score. Um score alto com esperados copiados da saída mede sensibilidade, não correção. Confira a origem de cada valor esperado.
>
> **[Aviso de Erro Comum]** Aceitar a verificação da skill como a sua. A skill é o agente conferindo o próprio trabalho. A evidência é a suíte na sua tela e o diff lido por você.

As seis evidências, reunidas

O fluxo mínimo terminou, e cada evidência do Capítulo 4 tem lugar e mecanismo:

- **Requisito esclarecido:** a spec em `docs/superpowers/specs/`, com os oito critérios identificados e o AC-08 num commit próprio.
- **Plano revisável:** o plano em `docs/superpowers/plans/`, auditado contra os IDs, num commit.
- **Teste falhando antes do código:** os vermelhos de cada tarefa, com o motivo lido, mais as sabotagens que mostraram cada teste pegando o defeito plausível.

- **Teste passando depois:** os verdes, tarefa por tarefa, com a suíte de caracterização verde e intocada do começo ao fim.
- **Diff revisado por você:** o diff da branch, conferido com o checklist acima.
- **Resultado verificado por você:** `Tests  10 passed (10)` no seu terminal.

Sob as seis está a linha de base caracterizada, que não é uma sétima evidência, mas é o que permitiu mexer no legado sabendo que o extrato continua cobrando o mesmo.

Compare com o fim do Capítulo 1. Lá, o agente encerrou com "Os testes devem passar", e havia três falhas escondidas. Aqui, cada uma foi barrada no seu momento: a busca pelo código e o snapshot, na spec; a tarifa divergente, num vermelho da caracterização e do AC-04; e o "devem passar" virou uma contagem na sua tela. O Capítulo 8 trata do que muda quando a história cresce e de como transformar cada uma dessas conferências num gate de pipeline que ninguém pula.

Resumo

- Prova por Falha: um teste só prova algo se você o viu falhar pelo motivo certo; falhar (valor divergente) é diferente de quebrar (o teste nem rodou).
- Mutação mede se a suíte distingue defeitos pequenos e plausíveis; a hipótese do programador competente e o efeito de acoplamento justificam o método, e ele descreve bem os erros de um agente.
- Mutantes equivalentes exigem análise registrada; e um score alto não prova o oráculo: esperados copiados da saída matam mutantes e certificam o defeito.
- Quebrar o código de propósito é a mutação manual; no legado, o vermelho da caracterização é alarme, e quem recua é a mudança.
- O Portão de Verificação Independente exige evidência fresca, completa e independente; a skill é só o primeiro grau.
- A evidência final é sua: a suíte rodada no seu terminal e o diff revisado com o checklist sênior.

Capítulo 8 — Escala, limites e próximos passos

Parte III — Além do primeiro teste

A Parte II rodou o fluxo mínimo numa sessão, uma tarefa por vez, com você lendo cada vermelho. Esse arranjo não escala: depende de um humano observando tudo ao vivo. Quando o volume cresce, delegar e isolar viram decisões de arquitetura, a evidência chega de segunda mão e o pipeline passa a ser o verificador que não depende nem do agente nem da sua atenção. No fim, ficam os limites que nenhuma dessas peças remove.

Escalar é isolar estado

O Capítulo 2 mostrou que uma sessão longa é estado acumulado e não auditável. Num plano de dez tarefas, a sétima é executada por um modelo que carrega, já compactados, os logs das seis anteriores, os arquivos que leu para elas e decisões que já não valem. Escalar não é rodar mais rápido; é impedir que o estado de uma unidade de trabalho contamine a seguinte. Os dois padrões de escala atacam dois tipos de estado.

Delegação com Contexto Limpo: isolar o estado do modelo Na Delegação com Contexto Limpo, implementada pela `subagent-driven-development`, o agente principal deixa de implementar e passa a coordenar. Para cada tarefa do plano, ele inicia um subagente com um contexto curado: a spec, o plano e o texto da tarefa. Nada mais. O subagente implementa com a Prova por Falha, devolve um relatório e termina. O coordenador guarda apenas o plano e os relatórios, nunca os logs.

É um escopo léxico aplicado ao modelo. Cada tarefa começa com a instrução relevante numa janela quase vazia, onde a atenção funciona melhor, e nenhuma suposição vaza de uma tarefa para outra pela conversa. O que atravessa as fronteiras atravessa por arquivo versionado: spec, plano e commits. Daí um critério de projeto: se o subagente da tarefa quatro precisa de algo que só existe na conversa da tarefa três, o que está incompleto é o plano.

A skill acrescenta dois revisores por tarefa, nesta ordem:

- **Conformidade com a spec:** o código faz o que a tarefa pede, nem menos, nem mais. Um critério de aceite esquecido reprova aqui, e

também uma função ou um parâmetro que ninguém pediu.
- **Qualidade:** só depois, a revisão olha nomes, duplicação, acoplamento e a qualidade dos testes.

A ordem é questão de custo: revisar a qualidade de um código que não cumpre a spec é polir a coisa errada. E os dois revisores também são modelos. A aprovação deles é um filtro barato, não um gate: reduz o que chega até você, sem substituir nenhuma evidência.

Quando a CLI não oferece subagentes, a `executing-plans` roda o plano em lotes na própria sessão, com pontos de revisão entre eles. É o mesmo princípio com um isolamento mais fraco: você troca a janela limpa por paradas obrigatórias.

Isolamento de Espaço de Trabalho: isolar o estado do sistema de arquivos O subagente isola o que o modelo sabe. Ele não isola o que o modelo toca. O Isolamento de Espaço de Trabalho, implementado pela `using-git-worktrees`, cuida da segunda parte: o trabalho acontece num worktree, com diretório, índice e `HEAD` próprios, ligado a uma branch própria e compartilhando só o banco de objetos do repositório.

A analogia é a do isolamento de processos. Cada worktree é um espaço de endereçamento próprio, e a única interface entre ele e a sua cópia de trabalho é uma branch. O limite também é o mesmo: o isolamento cobre arquivos, não o ambiente. Portas de rede, banco de dados local, caches globais e variáveis de ambiente continuam compartilhados. Dois worktrees rodando testes de integração contra o mesmo banco não estão isolados. Estão em condição de corrida.

Quando paralelizar A regra é a de concorrência de sempre: duas tarefas só rodam juntas quando não compartilham estado mutável. No plano, isso vira três perguntas:

- As duas tarefas editam algum arquivo em comum, incluindo lockfile, migrações e código gerado?
- Uma consome uma interface que a outra cria ou altera?
- Disputam algum recurso fora do repositório, como um schema, uma porta ou uma fila?

Qualquer "sim" serializa as duas. A `dispatching-parallel-agents` existe para o caso em que as três respostas são "não". Na dúvida,

serialize: sequência custa tempo; conflito custa um merge que nenhum agente entende.

> **[Exemplo]** No plano do ranking, nenhuma tarefa é paralelizável. A Task 1 só pode começar com a caracterização da Task 0 verde, porque é ela que prova que a extração não mudou o legado. A Task 3 consome o `searchBanks` da Task 2 e edita o mesmo arquivo. Um plano curto e coeso costuma ser sequencial, e isso é um bom sinal.

Delegue só um plano revisado, com tarefas que começam por um teste, nomeiam os arquivos e têm critério de pronto. Delegar um plano ambíguo multiplica a ambiguidade.

A evidência de segunda mão

Na Parte II, o "teste falhando antes do código" era uma saída que você leu. Com subagentes, ele chega dentro de um relatório, e um relatório é saída do agente: pelo princípio de zero-trust, uma alegação até ser conferida por um mecanismo que não dependa de quem a escreveu. Há três formas de fazer isso:

- **Exigir a saída literal.** Um relatório que diz "teste escrito, falhou e depois passou" sem mostrar a linha de asserção e o motivo volta para o subagente. "Expected 3 to be 1" é evidência; "o teste falhou como esperado" não é.
- **Tornar a ordem auditável.** Se o plano pedir um commit com o teste falhando antes do commit da implementação, o histórico da branch registra a Prova por Falha. Um checkout desse commit reproduz o vermelho a qualquer momento.
- **Delegar a sabotagem a uma máquina.** Quebrar o código de propósito, como no Capítulo 7, é a versão manual do teste de mutação. Em escala, quem quebra o código é o Stryker no pipeline. Se você não viu o teste nascer vermelho, uma ferramenta precisa tentar fazê-lo falhar. Ela prova sensibilidade, não oráculo: a origem de cada esperado ainda se confere no diff, contra a spec.

As duas evidências finais não mudam de dono: a suíte rodada por você sobre a branch do worktree e o diff contra a branch principal lido por você.

CI/CD: o pipeline como guardião zero-trust

A suíte que você rodou e o diff que você conferiu têm um ponto fraco comum: foram produzidos num ambiente que o agente também tocou. O pipeline é o único verificador que roda num ambiente limpo, com configuração que o agente não altera sem aprovação humana, e que não se cansa na décima história do dia. O Capítulo 4 associou cada evidência a um gate; aqui está o pipeline inteiro. Ele não substitui as seis evidências, porque o vermelho anterior ao código não existe num pipeline, mas torna as outras inegociáveis.

Os gates **1. Suíte completa, typecheck e lint.** O mínimo. O typecheck roda como gate próprio (`tsc --noEmit`) porque é a primeira defesa contra API inventada: um método que não existe na versão instalada não compila, por mais plausível que pareça no diff.

2. Integridade dos testes. Um agente sob pressão para "fazer passar" tem dois atalhos baratos: isolar o teste que interessa com `.only` e desligar o que incomoda com `.skip`. O primeiro o Vitest já barra, porque `allowOnly` é falso quando a variável `CI` está definida, e um `.only` esquecido faz a execução falhar. O segundo exige um gate explícito sobre o diff, que reprova qualquer `.skip` ou `.todo` adicionado na branch. Um teste desligado é um gate que alguém removeu sem pedir revisão.

3. A caracterização como check obrigatório. A suíte de caracterização do legado roda num job próprio, marcado como status check obrigatório na proteção da branch. Separada, ela aparece com nome próprio no PR, porque o vermelho ali tem outro significado: não "teste quebrado", e sim "o comportamento de produção mudou". Os arquivos dessa suíte ficam sob CODEOWNERS, para que nenhuma alteração neles entre sem a aprovação de quem responde pelo consumidor.

4. Rastreabilidade entre spec e teste. O `scripts/check-spec.mjs` recebe o caminho de uma spec e falha em dois casos: falta uma das quatro seções obrigatórias (Goal, Requirements, Acceptance criteria e Out of scope) ou algum identificador `AC-nn` da spec não aparece em nenhum arquivo `*.test.ts` versionado. O código completo está no Apêndice D.

```
node scripts/check-spec.mjs \
  docs/superpowers/specs/bank-ranking.md
```

[Exemplo] Na primeira execução real, o script falhou:

```
no test for AC-07
```

A suíte de caracterização cobria o AC-07, mas não o citava: o critério estava protegido sem ser rastreável. A correção foi pôr o identificador no `describe` da caracterização. Depois disso, o script imprimiu o caminho da spec seguido de `8 criteria traced`.

O gate confere a presença do identificador, não a semântica do teste: um teste chamado `AC-04` pode ser tautológico e passar. O gate seguinte cobre parte disso, a sensibilidade; a correção do oráculo continua dependendo de alguém conferir que o esperado veio da spec.

5. Mutação incremental. O Stryker mede se os testes distinguem o código certo de variações erradas dele. `--incremental` reaproveita os resultados da execução anterior, guardados em cache, e só reavalia os mutantes afetados pela mudança. O gate é o `thresholds.break`: abaixo dele, o processo termina com erro.

```
{
  "testRunner": "vitest",
  "mutate": ["src/**/*.ts", "!src/**/*.test.ts"],
  "thresholds": {
    "high": 90, "low": 80, "break": 80
  }
}
```

O limiar não é 100, porque mutantes equivalentes existem. No código final do exemplo, o Stryker gerou 52 mutantes e matou 51. O sobrevivente troca `toLowerCase()` por `toUpperCase()` numa normalização aplicada aos dois lados da comparação, um caso real de mutante equivalente. O score de 98,08 passa, mas todo sobrevivente novo exige leitura: ou é equivalente e fica documentado, ou é um teste que falta.

6. Branch protegida e revisão humana obrigatória. Merge só por PR, com os checks acima obrigatórios e pelo menos uma aprovação humana. Specs e planos entram no mesmo PR que o código e são revisados como código.

7. CODEOWNERS sobre o que define o comportamento. A caracterização, as specs e a própria configuração do pipeline têm donos. Com a opção de exigir a revisão dos code owners ligada na proteção da branch, ninguém aprova uma mudança nesses arquivos no mesmo PR em que ela serve de atalho:

```
*.characterization.test.ts  @acme/payments
docs/superpowers/specs/     @acme/payments
.github/                    @acme/platform
scripts/check-spec.mjs      @acme/platform
```

A última linha não é detalhe. Um gate que o PR pode editar não é um gate.

O agente sem credenciais O último controle é o que fica fora do alcance do agente. O ambiente dele não tem token de push nem credencial de deploy: ele produz uma branch local e um diff, e o envio é seu. O deploy parte do pipeline, depois do merge, por um ambiente protegido com aprovação própria; nos jobs de PR, o token do pipeline só lê. O agente propõe, os gates verificam, o humano aprova e o pipeline publica. Nenhum elo pode, ao mesmo tempo, escrever o código e publicá-lo.

Revisão Independente e o fechamento da branch

A Revisão Independente, implementada pela `requesting-code-review`, entrega a um revisor com contexto novo a spec, o plano e o intervalo de commits da branch, mas não a conversa que produziu o código. O modelo que escreveu o código racionaliza as próprias decisões, e um revisor que herda o contexto herda os pontos cegos. A revisão chega antes da sua leitura do diff, que você faz com os apontamentos na mão.

Os apontamentos também são saída de modelo. A `receiving-code-review` existe para tratá-los como hipóteses: cada um é verificado tecnicamente antes de ser aplicado, sem concordância por reflexo e sem rejeição por reflexo.

A `finishing-a-development-branch` fecha o trabalho. Ela começa rodando os testes e, se algum falhar, para sem oferecer nenhuma opção de integração. Com a suíte verde, oferece exatamente quatro caminhos:

1. fazer merge na branch principal, localmente;
2. enviar a branch e abrir um pull request;
3. manter a branch como está, para decidir depois;
4. descartar o trabalho, o que exige digitar `discard`.

A escolha é sua, e a regra do Capítulo 2 continua de pé: o push é uma decisão humana, tomada depois do diff revisado. Com o agente sem

credenciais, a opção 2 é executada por você, por construção, e não por disciplina.

Limites

Custo em tokens e tempo Cada padrão cobra: perguntas, um plano para ler, um contexto montado e duas revisões por tarefa, minutos de mutação no pipeline. Em tokens, o custo cresce com o número de tarefas multiplicado pelo contexto curado, com spec e plano reenviados a cada subagente.

O critério de decisão é o custo de descobrir o erro tarde:

- **Fluxo completo** onde há regra de negócio com casos de borda, contrato consumido por outro módulo, dinheiro, permissão ou concorrência. Ali, como no Capítulo 1, o erro encontrado depois do merge custa mais que todas as perguntas do começo.
- **Fluxo dispensado** numa correção de uma linha, num texto de interface ou numa mudança cuja verificação é óbvia. Os gates de CI continuam valendo; o que se dispensa é a cerimônia, e não a verificação.

Cadeia de suprimentos das skills Uma skill é instrução executada com as suas permissões, e pode carregar uma leitura de arquivos alheios à tarefa, um script na própria pasta ou uma injeção de prompt num texto inofensivo. Vale a regra do Capítulo 3, agora com a disciplina de uma dependência com acesso ao terminal:

- leia o `SKILL.md` e os scripts antes de instalar e a cada atualização, como um diff de dependência;
- fixe a versão incorporando ao repositório a pasta revisada (vendoring), com a origem e o commit registrados, para que a skill só mude por PR;
- ponha as skills do projeto sob CODEOWNERS, como as specs.

Ao trocar a instalação seletiva pelo pacote completo, o Apêndice A lista o que entra.

Degradação de contexto e API inventada As duas falhas do Capítulo 2 persistem em qualquer escala. Contra a degradação de contexto, a defesa é tirar o estado da conversa: decisões em arquivo versionado,

sessões curtas por etapa que começam lendo spec e plano, subagentes com contexto curado, worktrees separados e critério de saída verificável fora da sessão. Uma regra combinada há duas horas e esquecida pelo modelo é inevitável; uma regra escrita na spec e conferida por um gate não depende da memória de ninguém.

A API inventada nasce do mesmo mecanismo que torna o modelo útil: ele gera o plausível, e o treino mistura versões. As defesas, em ordem de custo:

- **typecheck** como gate, que pega a maior parte;
- **vermelho lido**, porque um `is not a function` é um vermelho pelo motivo errado e não prova nada sobre o comportamento;
- **lockfile e documentação da versão instalada**, em que os tipos publicados no pacote instalado, e não a memória do modelo nem a documentação da versão mais recente, são a fonte da verdade;
- **revisão do diff**, conferindo cada chamada a uma biblioteca pouco familiar.

A skill orienta, não garante Uma skill aumenta a chance de o processo ser seguido, sem torná-la certa. O modelo pode pular uma etapa, ler uma instrução de outro jeito, deixar de acionar a skill ou declarar cumprido um passo que não cumpriu, e mais agentes são mais lugares onde isso acontece. Por isso todo controle deste capítulo está fora do agente: a spec que você aprovou, o plano que você revisou, o vermelho que você leu, os gates que o pipeline executa e o diff que você conferiu. O PR sai com a sua aprovação, e a responsabilidade pelo que está nele continua sendo sua.

Código legado e integração

Caracterização como passo zero. Em código legado sem testes, a primeira tarefa do plano registra o que o código faz hoje, incluindo o que parece errado: é o que Michael Feathers chama de teste de caracterização, em *Working Effectively with Legacy Code*. Esses testes são o inverso da Prova por Falha: nascem verdes. Para provar que protegem algo, quebra-se o legado de propósito e confere-se o vermelho. O resultado é a linha de base caracterizada, um pré-requisito para que o vermelho seguinte signifique mudança de comportamento, e não acidente. Ela não é uma sétima evidência. A partir dela, a extração segue o padrão

strangler: a regra sai para uma função nova e testável, e o legado passa a delegar a ela, com a assinatura intacta.

Testes de contrato para o consumidor. Quando outro serviço depende do código legado, como o extrato depende do `getFee`, a assinatura e o comportamento são um contrato de interface. A caracterização registra o que o código faz. O contrato registra aquilo de que o consumidor depende, que costuma ser um subconjunto menor e mais estável. Quando a decisão "este comportamento é intencional" é tomada, o teste de caracterização vira teste de contrato e ganha um dono do lado do consumidor. Em contratos orientados ao consumidor, essas expectativas rodam no pipeline do provedor, e uma mudança que quebra o extrato falha antes do merge, não na fatura.

Dublês de serviço. Uma dependência externa entra por uma porta, uma interface pequena que o código consome, e os testes usam um dublê. É assim que um comportamento difícil de reproduzir, como o cache do pricing-service renovando entre duas leituras, vira um teste determinístico: o provider falso devolve uma versão nova da tabela a cada chamada, e o teste do snapshot único deixa de depender de sorte. O risco do dublê é divergir do serviço real em silêncio. A defesa é a mesma do parágrafo anterior: um teste de contrato contra o provedor verifica as suposições que o dublê codifica. O teste contra o serviço real fica para um ambiente separado.

Numa história real

O próximo passo é sair do exemplo. Escolha uma história do seu backlog e rode o fluxo mínimo do começo ao fim, conferindo as seis evidências: requisito esclarecido, plano revisável, teste falhando antes do código, teste passando depois, diff revisado por você e resultado verificado por você.

Dois critérios para a primeira:

- **Lógica com casos de borda:** faixas, limites, arredondamento, ordenação, concorrência. É onde a Elicitação Guiada de Requisitos encontra lacunas e onde um vermelho pega um erro de verdade.
- **Escopo de uma a três tarefas,** numa sessão, com você lendo cada vermelho. Subagentes, worktrees e paralelismo ficam para quando o formato das tarefas já for confiável.

Em paralelo, comece pelo pipeline. Typecheck, integridade dos testes e rastreabilidade da spec custam uma tarde e valem para todo código que entrar depois, de agente ou de gente. A mutação incremental vem em seguida, com um `break` baixo que sobe à medida que os sobreviventes são lidos.

> **[Nota]** Quando uma etapa não couber na sua história, anote por quê antes de pular. Se o mesmo motivo voltar em várias histórias, ele é material para uma skill do projeto, versionada no repositório e sob CODEOWNERS.

Os padrões sobrevivem às ferramentas

Os comandos deste livro vão mudar; os padrões, não. Elicitação Guiada de Requisitos, Decomposição Verificável, Prova por Falha e Portão de Verificação Independente formam o fluxo mínimo. Na escala, somam-se Delegação com Contexto Limpo, Isolamento de Espaço de Trabalho e Revisão Independente, e em código legado a Caracterização antes da Mudança vem antes de todos. Sem o superpowers, implemente cada padrão com o que houver: uma skill, um template de plano, um gate no pipeline. O que define o padrão é a evidência que ele deixa, não a ferramenta que a produz.

O livro começou com um agente rápido que entregou três falhas escondidas atrás de "os testes devem passar". O método não deixou o agente mais esperto. Ele mudou o que fica para trás a cada etapa: um arquivo revisado, uma saída lida, um gate que não negocia. Skill não torna o agente infalível; torna o trabalho dele verificável.

Resumo

- Escalar é isolar estado. A Delegação com Contexto Limpo isola o estado do modelo, com spec, plano e tarefa como único contexto, e o Isolamento de Espaço de Trabalho isola o sistema de arquivos, como processos. Paralelize só tarefas sem estado mutável compartilhado.
- A revisão em dois estágios, primeiro a conformidade com a spec e depois a qualidade, é um filtro, e não um gate. Com subagentes, a evidência chega de segunda mão: exija a saída literal, torne a ordem auditável no histórico e deixe a sabotagem para o teste de

mutação.

- O pipeline é o guardião zero-trust: suíte, typecheck e lint; integridade contra `.only` e `.skip`; caracterização como check obrigatório; `check-spec.mjs` para rastrear cada critério; Stryker incremental com `thresholds.break`; branch protegida e CODEOWNERS sobre specs, caracterização e o próprio pipeline.
- O agente não tem credenciais de push nem de deploy. A `finishing-a-development-branch` oferece quatro saídas, e o push é sempre seu.
- Os limites continuam: custo em tokens e tempo, cadeia de suprimentos das skills, degradação de contexto e API inventada. A skill orienta, não garante; por isso todo controle fica fora do agente.
- Em código legado, a caracterização é o passo zero, o contrato do consumidor ganha teste próprio e as dependências externas entram por portas com dublês verificados por contrato.

Apêndice A — Mapa das skills

Cada skill do superpowers é tratada neste livro como implementação de referência de um padrão. O mapa abaixo liga cada uma ao seu padrão canônico, quando há um, e diz quando ela entra. O pacote muda com o tempo: skills entram, saem e mudam de nome. Confira a lista na sua CLI antes de chamar qualquer uma.

Fluxo mínimo

- `brainstorming` — **Elicitação Guiada de Requisitos.** Antes de qualquer trabalho novo: explora o repositório, pergunta uma coisa de cada vez, propõe abordagens com trade-offs e grava a spec aprovada. Evidência: requisito esclarecido.
- `writing-plans` — **Decomposição Verificável.** Com a spec aprovada e nenhum código: tarefas pequenas, com arquivos explícitos, teste inicial e comando de verificação. Evidência: plano revisável.
- `test-driven-development` — **Prova por Falha.** Antes de qualquer código de produção: o teste vem primeiro e falha pelo motivo certo. Evidências: teste falhando antes do código e teste passando depois.
- `verification-before-completion` — **Portão de Verificação Independente.** Antes de qualquer afirmação de "pronto", "corrigido" ou "passando": o comando roda agora e a saída é lida. Evidência: resultado verificado.

Escala

- `subagent-driven-development` — **Delegação com Contexto Limpo.** Um subagente por tarefa, com spec, plano e tarefa como único contexto, e revisão em dois estágios: conformidade com a spec, depois qualidade.
- `using-git-worktrees` — **Isolamento de Espaço de Trabalho.** Um worktree e uma branch por linha de trabalho; isola arquivos, não o ambiente.
- `executing-plans` — sem padrão próprio; variação da Delegação com Contexto Limpo para CLIs sem subagentes. Executa o plano em lotes, na mesma sessão, com pontos de revisão.
- `dispatching-parallel-agents` — sem padrão próprio; aplica a Delegação com Contexto Limpo a tarefas sem estado mutável com-

partilhado, em paralelo.

Revisão e entrega

- `requesting-code-review` — **Revisão Independente.** Um revisor sem a conversa da implementação compara a branch com a spec e o plano, antes da sua leitura do diff.
- `receiving-code-review` — complemento da Revisão Independente: cada apontamento é verificado tecnicamente antes de ser aplicado.
- `finishing-a-development-branch` — fechamento: roda a suíte e, só com ela verde, oferece as quatro saídas (merge local, PR, manter, descartar). O push é decisão sua.

Depuração

- `systematic-debugging` — sem padrão canônico no livro. Diante de um bug ou de um teste falhando, investiga a causa antes de propor correção, o que aplica ao diagnóstico a mesma recusa ao plausível que o Portão de Verificação Independente aplica ao "pronto".

Skills sobre as skills

- `using-superpowers` — carregada no início da sessão; faz o agente procurar a skill aplicável antes de responder.
- `writing-skills` — para criar, editar e testar skills de projeto, como a `fee-rules` do Capítulo 3.
- `diagnosing-superpowers` — para investigar uma sessão que deu errado, como uma skill não acionada ou um trabalho repetido.

Sem skill própria

- **Caracterização antes da Mudança.** O passo zero em código legado não tem skill no superpowers. Exija-o numa skill de projeto ou no template do plano, e torne a suíte resultante um check obrigatório no pipeline, como no Capítulo 8.

Qualquer skill fora do superpowers segue a regra de cadeia de suprimentos do Capítulo 3: revisão integral antes de instalar, versão fixada no repositório e diff relido a cada atualização.

Apêndice B — Glossário

Agente — Programa que usa um modelo de linguagem para decidir e executar ações, como ler arquivos e rodar comandos. Neste livro, um agente é sempre a soma de um modelo com um harness.

Alucinação de API — Chamada a uma função, opção ou parâmetro plausível que não existe na versão instalada, ou que existiu em outra. É estrutural: o modelo gera o provável, o treino mistura versões e APIs internas não estão no treino. Defesas: typecheck, vermelho lido, lockfile e tipos da versão instalada, revisão do diff.

Cadeia de suprimentos (supply chain) — Conjunto de código e instruções de terceiros que entra no seu ambiente. Skills, plugins e servidores MCP são dependências privilegiadas: dirigem um agente com shell e os seus privilégios. Pede revisão integral, versão fixada e reavaliação a cada atualização.

Caracterização (teste de) — Teste que registra o que um código faz hoje, incluindo o que parece errado, sem julgar se está certo (Michael Feathers, *Working Effectively with Legacy Code*). Nasce verde, ao contrário do TDD, e só prova que protege algo quando se quebra o código de propósito e se vê o vermelho. O conjunto forma a *linha de base caracterizada*: uma suíte que congela o contrato atual, deve continuar verde sem alteração durante toda a mudança e é pré-requisito das seis evidências, e não uma sétima.

Caracterização antes da Mudança (*Characterize Before Change*) — Padrão do passo zero em código legado: antes de qualquer mudança, testes de caracterização fixam o comportamento atual, para que o vermelho seguinte signifique mudança de comportamento, e não acidente. Não tem skill própria no superpowers.

Compactação — Liberação de espaço na janela de contexto feita pelo harness quando ela enche: truncar saídas antigas, descartar turnos ou resumir o histórico com o próprio modelo. O resumo não avisa o que perdeu, e decisões de borda costumam ser as primeiras a sumir.

Constituição — No Spec Kit, documento com os princípios do projeto que toda spec deve respeitar. A spec muda a cada história; a constituição vale para todas.

Contexto — Tudo o que o modelo enxerga numa chamada: prompt de

sistema, definições de ferramentas, instruções, metadados das skills e o histórico de pedidos, leituras e saídas. É finito, medido em tokens, e degrada antes de encher.

Contrato de interface — Assinatura e comportamento observável de que um consumidor depende, como o `getFee` legado para o serviço de extrato. A caracterização registra tudo o que o código faz; o contrato, só o que o consumidor usa. Testes de contrato orientados ao consumidor rodam essas expectativas no pipeline do provedor.

Costura (*seam*) — No vocabulário de Feathers, ponto onde o comportamento pode ser trocado sem editar o código sob teste. Em código orientado a objetos, a costura mais limpa é uma porta recebida por injeção de dependência. Critério que depende de rede, relógio ou estado global sem costura não é testável de forma determinística.

Critério de aceite — Caso concreto, com entrada e resultado esperado, que mostra que um requisito da spec foi cumprido. Com um identificador (`AC-nn`), torna-se rastreável até o teste que o cobre.

Decomposição Verificável (*Verifiable Decomposition*) — Padrão em que a spec vira um plano de tarefas pequenas, cada uma com arquivos explícitos, um teste que começa falhando e o comando que prova o resultado. Evidência: plano revisável. Referência: `writing-plans`.

Degradação de contexto — Perda de fidelidade de uma sessão longa: compactação que descarta detalhes, recuperação pior do meio do contexto (*lost in the middle*), saídas ruidosas que competem com as instruções e instruções antigas que perdem peso. Torna a sessão longa um estado acumulado e não auditável.

Delegação com Contexto Limpo (*Clean-Context Delegation*) — Padrão em que um coordenador entrega cada tarefa a um subagente com contexto curado (spec, plano e tarefa), recebe só o resultado e revisa antes da próxima. Isola o estado do modelo. Referência: `subagent-driven-development`.

Desenvolvimento verificável — Forma de trabalhar em que cada etapa deixa uma evidência que você consegue ver e que não depende da palavra do agente: antes, durante e depois do código. É o método deste livro.

Divulgação progressiva (*progressive disclosure*) — Forma como as skills

gastam o orçamento de contexto, em três níveis: metadados sempre carregados, corpo ao acionar e arquivos de apoio sob demanda. Por isso a descrição é a interface da skill.

Dublê de serviço — Implementação falsa de uma dependência externa, plugada por uma porta, que torna determinístico um comportamento difícil de reproduzir com o serviço real. Diverge do serviço em silêncio se não for verificado por um teste de contrato.

Elicitação Guiada de Requisitos (*Guided Requirements Elicitation*) — Padrão em que, antes de qualquer código, o agente explora o repositório, pergunta uma coisa de cada vez, propõe abordagens com trade-offs e só grava a spec depois da sua aprovação. Evidência: requisito esclarecido. Referência: `brainstorming`.

Evidência — Artefato concreto que mostra que uma etapa foi cumprida, como uma spec aprovada, um teste visto falhando ou um diff revisado, e não a palavra do agente. O método produz seis: requisito esclarecido, plano revisável, teste falhando antes do código, teste passando depois, diff revisado por você e resultado verificado por você.

Ferramenta — Ação que o harness oferece ao modelo, como ler um arquivo, editar código ou rodar um comando. O modelo decide quando usar; o harness executa, sob a política de permissões.

Fluxo mínimo — As quatro skills usadas neste livro, uma por etapa: `brainstorming` (esclarecer), `writing-plans` (planejar), `test-driven-development` (vermelho/verde) e `verification-before-completion` (verificar).

Fronteira de confiança — Linha entre um componente não confiável e o que ele afeta. No agente, o modelo é o principal não confiável, e a permissão é o ponto de aplicação da política entre o que ele propõe e o que acontece. É porosa por composição de permissões, por fadiga de aprovação e porque instrução não é controle.

Frontmatter — Cabeçalho YAML no topo de um `SKILL.md`, com o nome (`name`) e a descrição (`description`) da skill.

Gate — Verificação automatizada e bloqueante no pipeline, que barra o merge quando falha: suíte, typecheck, rastreabilidade, mutação, revisão obrigatória. Cada evidência tem um gate correspondente. O gate não substitui a evidência local: pega o mesmo erro mais tarde, quando a

etapa foi pulada.

Harness — Programa que envolve o modelo: monta o contexto, oferece ferramentas, aplica as permissões e executa o loop de ações. É a parte determinística do agente. O GitHub Copilot CLI e o Claude Code são exemplos.

Hipótese do programador competente — Premissa do teste de mutação (DeMillo, Lipton e Sayward, 1978): programas escritos por profissionais competentes estão perto do correto, e os defeitos restantes são desvios pequenos. Junto com o *efeito de acoplamento*, segundo o qual testes que detectam os defeitos simples também detectam a maioria dos complexos, justifica medir a suíte com mutantes de primeira ordem. Descreve bem o código de um agente: plausível na estrutura, errado no operador ou no limite.

Hook — Comando que o harness ou um plugin dispara em eventos do loop, como antes de uma edição ou ao fim de um turno, sem passar pelo modelo. Por ser determinístico, é lugar para controles que precisam valer sempre; vindo de terceiros, é código com os seus privilégios.

Implementação de referência — Implementação concreta de um padrão, usada como modelo e não como definição. Neste livro, cada skill do superpowers é a implementação de referência de um padrão de engenharia com IA; o padrão continua valendo em outra CLI, outro pacote ou um processo do próprio time.

Injeção de dependência — Entrega das dependências a um componente por fora, em geral pelo construtor, em vez de ele importá-las ou lê-las de estado global. É o que permite plugar um dublê numa porta.

Injeção de prompt (*prompt injection*) — Instrução embutida em dado que o agente lê, como um comentário de código, uma issue ou um README, que o modelo trata com o mesmo status do seu pedido. Na forma indireta, quem controla o dado controla parte do agente.

Isolamento de Espaço de Trabalho (*Workspace Isolation*) — Padrão em que cada linha de trabalho ganha um worktree e uma branch próprios. Isola o estado do sistema de arquivos, como o isolamento de processos, mas não o ambiente (portas, bancos, caches). Referência: `using-git-worktrees`.

Lost in the middle — Efeito descrito por Liu et al. (2023): modelos re-

cuperam pior a informação do meio de contextos longos do que a do começo ou do fim. Numa sessão de agente, o meio é onde ficam as decisões já tomadas.

MCP — Sigla de *Model Context Protocol*, padrão aberto para ligar serviços externos a um agente como ferramentas. Um servidor MCP é código de terceiros com acesso de execução ou um token seu.

Modelo — Modelo de linguagem: função sem estado sobre o contexto, que recebe o contexto inteiro a cada chamada e devolve uma continuação. Sozinho, não lê arquivos nem roda comandos; a "memória" da sessão é o histórico que o harness reenvia.

Mutação (teste de) — Técnica que gera variações pequenas do código (mutantes), trocando operadores relacionais e limites, negando condicionais ou removendo chamadas, e roda a suíte contra cada uma. Mutante que faz algum teste falhar está morto; o que passa sobreviveu. O *mutation score* é a fração de mortos sobre os avaliados. Mede a sensibilidade da suíte, não a correção do oráculo. Fundamentos na hipótese do programador competente. Neste livro, a ferramenta é o Stryker, com `thresholds.break` como gate de CI.

Mutante equivalente — Mutante que não muda o comportamento observável do programa e, por isso, nenhum teste consegue matar. Exemplo real: trocar `toLowerCase()` por `toUpperCase()` numa normalização aplicada aos dois lados da comparação. É o motivo de o limiar de mutação não ser 100.

Oráculo (de teste) — A fonte do valor esperado num teste. Precisa ser independente do código testado: o critério de aceite da spec ou o comportamento caracterizado de um consumidor em produção. Um oráculo derivado da implementação produz teste tautológico, mesmo com score de mutação alto.

Padrões de engenharia com IA — Formas recorrentes de restringir o agente para que ele produza artefatos verificáveis. Cada um nomeia um problema, um mecanismo e a evidência que deixa. Os oito canônicos deste livro: Elicitação Guiada de Requisitos, Decomposição Verificável, Prova por Falha, Portão de Verificação Independente, Delegação com Contexto Limpo, Isolamento de Espaço de Trabalho, Revisão Independente e Caracterização antes da Mudança.

Permissão — Autorização que o harness exige antes de uma ação com efeito, como rodar um comando ou editar um arquivo. É a fronteira de confiança entre o modelo e o sistema; vale tanto quanto a leitura de quem aprova e não substitui a ausência de capacidade, como um agente sem credencial de push.

Plano — Documento que divide uma spec em tarefas pequenas e ordenadas, cada uma com arquivos, teste inicial e comando de verificação. Revisado antes de qualquer código e versionado com ele.

Plugin — Pacote que a CLI instala e atualiza de uma vez, com skills, comandos, hooks e configurações. O superpowers pode ser instalado como plugin.

Porta (*port*) — Interface pequena, definida pelo código que a consome, por onde entra uma dependência externa, como o `FeeTableProvider` para o pricing-service. Recebida por injeção de dependência, é a costura que permite trocar o serviço real por um dublê nos testes.

Portão de Verificação Independente (*Independent Verification Gate*) — Padrão em que nenhuma afirmação de conclusão é aceita sem o comando que a prova executado agora, com a saída lida. Evidência: resultado verificado. Referência: `verification-before-completion`.

Prova por Falha (*Fail-First Proof*) — Padrão em que o teste vem antes do código e precisa falhar pelo motivo certo; só então vem o código mínimo para passar. Evidências: teste falhando antes do código e teste passando depois. Referência: `test-driven-development`.

Rastreabilidade — Ligação verificável entre um requisito e o teste que o cobre. Neste livro, cada critério de aceite tem um identificador `AC-nn`, e o `check-spec.mjs` falha no CI se algum deles não aparecer em nenhum arquivo de teste. Confere a presença, não a qualidade do teste.

Revisão Independente (*Independent Review*) — Padrão em que a revisão é feita por um contexto que não participou da implementação, contra a spec e o plano, antes da sua leitura do diff. Referência: `requesting-code-review`.

SDD (Spec-Driven Development) — Engenharia de requisitos executável: a especificação é escrita e aprovada antes do código e guia o plano, os testes e a revisão, com critérios identificados ligados a testes por um vínculo que um script confere no CI. Cada artefato (spec, plano, tarefas,

código) é revisado antes do próximo.

Skill — Pasta com um `SKILL.md` que descreve um processo para o agente: frontmatter com nome e descrição e, abaixo, as instruções. Por ser arquivo, é versionada e revisável em PR. O acionamento pela descrição não é determinístico: o modelo decide se a skill se aplica, e pode errar.

Snapshot — Leitura única e imutável de um estado que pode mudar, usada do começo ao fim de uma operação, como a tabela de tarifas lida uma vez por requisição. Não confundir com o *snapshot test*, que compara a saída com um arquivo gravado e vira teste tautológico quando é aceito sem leitura.

Spec — Especificação do que deve ser construído, com quatro seções: objetivo, requisitos, critérios de aceite, cada um com um ID (`AC-nn`), e fora do escopo. Escrita antes do plano, aprovada por uma pessoa e revisada no PR como código.

Spec Kit — Kit aberto do GitHub para praticar SDD com um agente, publicado em `github/spec-kit`. Conduz cada etapa por um comando, começando pela constituição do projeto.

Strangler — Padrão de migração incremental em que a lógica sai do código antigo para um componente novo e testável, e o antigo passa a delegar a ele com a interface intacta, até poder ser removido. Neste livro, o `getFee` legado delega à função pura `feeFor`.

Subagente — Agente iniciado por outro agente para uma tarefa específica, com contexto próprio. Ao terminar, devolve só o resultado ao coordenador.

Superpowers — Pacote aberto de skills de engenharia de software, publicado em `obra/superpowers`. Neste livro, é a implementação de referência dos padrões; o Apêndice A mapeia o pacote.

TDD (Test-Driven Development) — Prática de escrever o teste antes do código de produção, no ciclo vermelho, verde e refatoração. Neste livro, o vermelho visto é tratado como evidência, e não como ritual.

Teste tautológico — Teste cujo valor esperado deriva da implementação, e não do requisito; passa por construção e congela o erro. Há duas famílias. Os *sem sensibilidade* (espelho por reuso, asserção sobre o mock

em vez do efeito, asserções fracas) deixam mutantes vivos, e a mutação os denuncia. Os *sem oráculo independente* (espelho por cópia, snapshot aceito sem leitura, esperado copiado da saída) matam mutantes e, ainda assim, certificam o defeito.

Token — Unidade em que o modelo lê e gera texto: fragmentos de palavra cujo tamanho varia com a língua e o tipo de conteúdo. Código, JSON e logs rendem mais tokens por linha do que prosa. A janela de contexto e o custo são medidos em tokens.

Tríade letal — Combinação descrita por Simon Willison (2025): acesso a dados privados, exposição a conteúdo não confiável e capacidade de comunicação externa. Um agente com as três pode ser induzido, por injeção de prompt, a ler um segredo e mandá-lo para fora.

Vendoring — Incorporar ao repositório a cópia revisada de uma dependência, registrando a origem e o commit de onde veio. Para skills de terceiros, é a forma mais simples de fixar a versão: a partir dali, a skill só muda por PR.

Vermelho/verde — Os dois estados de um teste no ciclo do TDD. Ver o vermelho, pelo motivo certo, antes do verde é a evidência de que o teste verifica algo.

Worktree — Cópia de trabalho adicional de um repositório git, com diretório, índice e `HEAD` próprios, ligada a outra branch e compartilhando o banco de objetos.

Zero-trust — Princípio aplicado neste livro ao desenvolvimento com IA: toda saída do agente, incluindo código, relatório e "pronto", é não confiável até ser verificada por um mecanismo que não dependa dele, como um teste visto falhando, um comando rodado por você ou um gate de CI.

Apêndice C — Equivalências entre as CLIs

Referência rápida dos comandos e pastas apresentados no Capítulo 3. Em cada bloco, a primeira forma é a do Claude Code e a segunda, a do Copilot CLI. As barras invertidas só quebram o comando para caber na página.

Instalar o pacote como plugin

Claude Code (dentro da sessão):

```
/plugin install superpowers@claude-plugins-official
```

Copilot CLI:

```
copilot plugin marketplace add \
  obra/superpowers-marketplace
copilot plugin install \
  superpowers@superpowers-marketplace
```

Instalar só algumas skills

O comando é o mesmo nas duas CLIs (Node.js 22 ou superior); muda o valor de `-a`: `claude-code` ou `github-copilot`. Com `-g`, a instalação vai para a pasta pessoal; sem ele, para o projeto atual.

```
npx skills add obra/superpowers -g \
  -a claude-code \
  -s brainstorming writing-plans \
  test-driven-development \
  verification-before-completion
```

Pastas de skills

Claude Code

- projeto: `.claude/skills/`;
- pessoal: `~/.claude/skills/`.

Copilot CLI

- projeto: `.github/skills/`, `.claude/skills/` ou `.agents/skills/`;
- pessoal: `~/.copilot/skills/` ou `~/.agents/skills/`.

A pasta `.claude/skills/` do projeto serve às duas CLIs. Num time que usa ambas, é o lugar das skills do projeto, versionadas e sob revisão.

Conferir e chamar

- *Claude Code*: `/` na sessão lista as skills.
- *Copilot CLI*: `/skills list`.

As duas CLIs leem as skills no início da sessão. Nas duas, a chamada explícita é uma barra, o nome da skill e o pedido:

```
/brainstorming <a história, como está no
backlog>
```

Instalada como plugin, a skill pode aparecer com o prefixo do pacote, como `/superpowers:brainstorming`. É a mesma skill.

> **[Nota]** Comandos conferidos em outubro de 2026. As CLIs mudam com frequência; na divergência, vale a documentação da versão instalada.

Apêndice D — Projeto do exemplo

Este é o `transfer-service` usado na Parte II, reduzido ao necessário para reproduzir cada etapa. O código de chegada é referência para comparar com o que o seu agente produzir, e não para copiar antes da hora: na prática, quem o escreve é o agente, conduzido pelas skills e pelos gates.

Estrutura e dependências

```
transfer-service/
  docs/specs/bank-ranking.md
  scripts/check-spec.mjs
  src/
    banks.ts
    fees/feePolicy.ts
    legacy/fees.ts
    legacy/fees.characterization.test.ts
    pricing/feeTable.ts
    ranking/bankRanking.ts
    ranking/bankRanking.test.ts
  package.json
  stryker.config.json
```

Os arquivos de `fees/` e `ranking/`, a caracterização, a spec e o script nascem durante a história. No repositório de referência, a spec fica em `docs/specs/` por simplicidade; gravada pela `brainstorming`, ela estaria em `docs/superpowers/specs/`. A configuração do Stryker está no Capítulo 8. As dependências de desenvolvimento:

```
{
  "devDependencies": {
    "@stryker-mutator/core": "^10.1.1",
    "@stryker-mutator/vitest-runner": "^10.1.1",
    "typescript": "^7.0.2",
    "vitest": "^5.0.5"
  }
}
```

Ponto de partida

O cadastro de bancos, `src/banks.ts`:

```
export type Bank = { code: string; name: string };

export const banks: Bank[] = [
  { code: "001", name: "Banco Azul" },
```

```
  { code: "012", name: "Banco Verde" },
  { code: "123", name: "Banco Âmbar" },
];
```

Os tipos da tabela de tarifas e a porta do pricing-service, `src/pricing/feeTable.

```
export type FeeRange = {
  lowFee: number;
  highFee: number;
};

export type FeeTable = {
  version: number;
  limit: number;
  fees: Record<string, FeeRange>;
};

export interface FeeTableProvider {
  current(): Promise<FeeTable>;
}
```

O legado original, `src/legacy/fees.ts`, sem testes e lendo a tabela global mutável que o serviço de extrato também usa:

```
import type { FeeTable } from "../pricing/feeTable";

// Loaded at startup and replaced by the nightly
// pricing sync. Used by the statement service.
let FEE_TABLE: FeeTable;

export function loadFeeTable(table: FeeTable) {
  FEE_TABLE = table;
}

export function getFee(
  code: string, amount: number
): number {
  const f = FEE_TABLE.fees[code];
  if (!f) throw new Error("unknown bank " + code);
  if (amount > FEE_TABLE.limit) return f.highFee;
  return f.lowFee;
}
```

Ponto de chegada

A política extraída, `src/fees/feePolicy.ts`. É uma função pura sobre um snapshot da tabela, com o limite inclusivo herdado do legado:

```
import type { FeeTable } from "../pricing/feeTable";
```

```
export function feeFor(
  table: FeeTable, code: string, amount: number
): number | undefined {
  const range = table.fees[code];
  if (!range) return undefined;
  return amount > table.limit
    ? range.highFee
    : range.lowFee;
}
```

O legado depois da Task 1, `src/legacy/fees.ts`, delegando à política (strangler), com a assinatura e a mensagem de erro intactas:

```
import { feeFor } from "../fees/feePolicy";
import type { FeeTable } from "../pricing/feeTable";

// Loaded at startup and replaced by the nightly
// pricing sync. Used by the statement service.
let FEE_TABLE: FeeTable;

export function loadFeeTable(table: FeeTable) {
  FEE_TABLE = table;
}

export function getFee(
  code: string, amount: number
): number {
  const fee = feeFor(FEE_TABLE, code, amount);
  if (fee === undefined) {
    throw new Error("unknown bank " + code);
  }
  return fee;
}
```

O serviço de ranking, `src/ranking/bankRanking.ts`, com a lista de bancos e o provider injetados e uma única leitura da tabela por requisição:

```
import type { Bank } from "../banks";
import { feeFor } from "../fees/feePolicy";
import type { FeeTableProvider }
  from "../pricing/feeTable";

export type RankedBank = Bank & {
  fee: number;
  tableVersion: number;
};

function normalize(text: string): string {
  return text
```

```
    .normalize("NFD")
    .replace(/\p{Diacritic}/gu, "")
    .toLowerCase();
}

export function searchBanks(
  banks: Bank[], term: string
): Bank[] {
  const t = normalize(term);
  return banks.filter((b) =>
    normalize(b.name).includes(t) ||
    b.code === term
  );
}

export class BankRankingService {
  constructor(
    private readonly banks: Bank[],
    private readonly tables: FeeTableProvider,
  ) {}

  async rank(
    term: string, amount: number
  ): Promise<RankedBank[]> {
    // One snapshot per request: a refresh in the
    // middle cannot mix two table versions.
    const table = await this.tables.current();
    const ranked: RankedBank[] = [];
    const found = searchBanks(this.banks, term);
    for (const bank of found) {
      const fee = feeFor(table, bank.code, amount);
      if (fee === undefined) continue;
      ranked.push({
        ...bank, fee, tableVersion: table.version,
      });
    }
    return ranked.sort((a, b) =>
      a.fee - b.fee ||
      a.name.localeCompare(b.name, "pt-BR")
    );
  }
}
```

A suíte de caracterização, `src/legacy/fees.characterization.test.ts`. O `AC-07` no `describe` é a correção que o `check-spec.mjs` exigiu na primeira execução:

```
import { beforeEach, describe, expect, it }
  from "vitest";
```

```
import { getFee, loadFeeTable } from "./fees";

// Pins what production does today. Not the spec:
// a red here means behavior changed.
describe("AC-07 legacy getFee, as is", () => {
  beforeEach(() => {
    loadFeeTable({
      version: 1,
      limit: 1000,
      fees: { "001": { lowFee: 2, highFee: 5 } },
    });
  });

  it("charges lowFee at exactly the limit", () => {
    expect(getFee("001", 1000)).toBe(2);
  });

  it("charges highFee above the limit", () => {
    expect(getFee("001", 1000.01)).toBe(5);
  });

  it("throws for an unknown bank", () => {
    expect(() => getFee("999", 10))
      .toThrow("unknown bank 999");
  });
});
```

Os testes do ranking, `src/ranking/bankRanking.test.ts`. O provider falso devolve uma versão nova da tabela a cada chamada, como um cache renovado entre duas leituras:

```
import { describe, expect, it } from "vitest";
import { banks } from "../banks";
import type { FeeTable, FeeTableProvider }
  from "../pricing/feeTable";
import { BankRankingService, searchBanks }
  from "./bankRanking";

const v1: FeeTable = {
  version: 1,
  limit: 1000,
  fees: {
    "001": { lowFee: 2, highFee: 5 },
    "012": { lowFee: 3, highFee: 4 },
    "123": { lowFee: 2, highFee: 6 },
  },
};

// Returns a newer table on every call, like a
```

```
// cache refreshed between two reads.
function refreshingProvider(): FeeTableProvider & {
  calls: number;
} {
  return {
    calls: 0,
    async current() {
      this.calls++;
      return { ...v1, version: this.calls };
    },
  };
}

const names = (list: { name: string }[]) =>
  list.map((b) => b.name);

describe("searchBanks", () => {
  it("AC-01 ignores case and accents", () => {
    expect(names(searchBanks(banks, "AZUL")))
      .toEqual(["Banco Azul"]);
    expect(names(searchBanks(banks, "ambar")))
      .toEqual(["Banco Âmbar"]);
  });

  it("AC-02 matches the exact code only", () => {
    expect(names(searchBanks(banks, "001")))
      .toEqual(["Banco Azul"]);
    expect(searchBanks(banks, "1")).toEqual([]);
  });

  it("AC-03 returns all banks for ''", () => {
    expect(searchBanks(banks, "")).toHaveLength(3);
  });
});

describe("BankRankingService", () => {
  it("AC-04 lowFee at the limit", async () => {
    const service = new BankRankingService(
      banks, refreshingProvider());
    const [azul] = await service.rank("001", 1000);
    expect(azul.fee).toBe(2);
  });

  it("AC-05 fee, then name", async () => {
    const service = new BankRankingService(
      banks, refreshingProvider());
    const ranked = await service.rank("", 500);
    expect(names(ranked)).toEqual([
      "Banco Âmbar", "Banco Azul", "Banco Verde",
```

```
    ]);
  });

  it("AC-06 uses one table snapshot", async () => {
    const provider = refreshingProvider();
    const service = new BankRankingService(
      banks, provider);
    const ranked = await service.rank("", 500);
    expect(provider.calls).toBe(1);
    const versions = ranked.map(
      (r) => r.tableVersion);
    expect(new Set(versions)).toEqual(new Set([1]));
  });

  it("AC-08 skips banks missing from the table",
    async () => {
      const { "123": _, ...fees } = v1.fees;
      const service = new BankRankingService(
        banks,
        { current: async () => ({ ...v1, fees }) },
      );
      const ranked = await service.rank("", 500);
      expect(names(ranked)).toEqual([
        "Banco Azul", "Banco Verde",
      ]);
    });
});
```

A spec, `docs/specs/bank-ranking.md`:

```
# Bank ranking by fee

## Goal
Help the customer pick the cheapest bank for
the transfer they are about to make.

## Requirements
- Search by bank name or bank code.
- Fees come from the same policy the statement
  service charges with.

## Acceptance criteria
- AC-01 "azul" and "AZUL" find Banco Azul;
  "ambar" finds Banco Âmbar.
- AC-02 "001" finds Banco Azul; "1" finds none.
- AC-03 "" (empty) finds all banks.
- AC-04 Banco Azul fee for 1000 is 2; for
  1000.01 is 5.
- AC-05 Order for 500: Âmbar, Azul, Verde.
- AC-06 One fee table snapshot per request.
```

```
- AC-07 Legacy getFee keeps its signature and
  its characterization suite passes unchanged.
- AC-08 A bank missing from the snapshot is
  left out of the ranking.

## Out of scope
- Fee exemptions and promotions.
- Changes to the statement service.
```

O gate de rastreabilidade, `scripts/check-spec.mjs`:

```
// Fails CI when a spec misses a required section
// or an acceptance criterion has no test.
import { readFileSync } from "node:fs";
import { execSync } from "node:child_process";

const [spec] = process.argv.slice(2);
const text = readFileSync(spec, "utf8");
const errors = [];

for (const h of ["Goal", "Requirements",
  "Acceptance criteria", "Out of scope"]) {
  if (!text.includes(`## ${h}`)) {
    errors.push(`missing section: ${h}`);
  }
}

const ids = new Set(text.match(/AC-\d+/g));
const tests = execSync(
  "git ls-files '*.test.ts'", { encoding: "utf8" }
).split("\n").filter(Boolean)
  .map((f) => readFileSync(f, "utf8")).join("\n");

for (const id of ids) {
  if (!tests.includes(id)) {
    errors.push(`no test for ${id}`);
  }
}

if (errors.length) {
  console.error(errors.join("\n"));
  process.exit(1);
}
console.log(`${spec}: ${ids.size} criteria traced`);
```

Com tudo no lugar, a suíte termina com `Tests  10 passed (10)` em dois arquivos de teste, o `check-spec.mjs` rastreia os oito critérios e o Stryker registra 52 mutantes, 51 mortos e um sobrevivente equivalente, com score de 98,08.

[Nota] O projeto foi conferido com Node 24, Vitest 5 e Stryker 10.1. Versões futuras podem mudar a saída, não o método.